

5 第五章 飞升入定：云原生部署

到第四章为止，游戏后端在逻辑上已经具备多机运行的结构：数据库分片承载存储容量，负载均衡分配玩家连接，协调服务器裁决跨服事件，Raft 集群保护关键状态。但这些组件要真正运行在多台服务器上，还需要回答一个工程问题：如何把它们稳定地交付、部署和维护？

本章所说的“上云”，是将应用部署到由云平台管理的计算、网络和存储资源上，使其能够面向互联网持续提供服务，并在实例增减、节点故障和环境差异下保持可交付、可调度和可恢复。部署对象不再是固定服务器上的若干进程，而是一组可能被复制、替换和重新放置的服务实例。

当系统只有一两台服务器时，运维人员可以逐台登录、手工安装依赖、逐个启动进程。一旦组件数量增多、服务器台数增加，手工操作的可靠性会迅速下降。具体表现为三类困难。第一，环境差异：同一份代码在开发机上能运行，换到另一台服务器后可能因为系统库版本、时区设置或目录结构不同而无法启动。第二，部署复杂度：大厅服、战斗服、Redis、PostgreSQL 和入口层分散在多台机器上，启动顺序、配置注入和网络互联都需要人工对齐。第三，运维脆弱性：扩容和故障恢复依赖人工响应，而高峰时段正是人最容易出错的时候。

本章按照“容器—编排—弹性运行”三大块展开。第一块讨论容器如何把代码、依赖和运行配置固化为可复制的交付物，解决“换一台机器还能否按同样方式运行”的问题。第二块讨论编排如何描述多个服务的启动、互联和副本关系，并把单机部署扩展到多机集群，解决“多个组件由谁统一部署和维护”的问题。第三块讨论系统运行之后，平台如何完成资源调度、自动伸缩、按需执行和故障自愈，解决“负载变化或实例故障时如何维持服务目标”的问题。三块内容依次把关注点从单个运行环境，推进到多服务拓扑，再推进到持续运行中的容量与故障。

5.1 容器：上云的虚拟化基础

容器部分先说明为什么同一份代码换机器后可能无法启动，再介绍镜像、容器、网络和数据卷等机制如何解决这一问题。这些机制共同支撑一条“构建一次、多处运行”的交付链路：镜像在开发环境构建完成，推送到仓库统一分发，再由目标机器拉取后直接运行。本节的重点在于理解容器为什么能把一次部署结果稳定地复制到其他机器，具体的 Docker 命令会在必要时作为示例出现。

5.1.1 为什么需要容器：环境一致性

一份程序能够正常运行，不只取决于源代码本身，还取决于它所处的运行环境。环境包括操作系统版本、动态链接库、内核接口、目录结构、用户权限、时区设置和字符编码等条件。当开发机和部署目标机器的环境不同时，同一份可执行文件可能在一边正常启动，在另一边无法运行。理解环境差异的来源，是理解容器、镜像不可变和运行时配置注入等后续机制的前提。

隐性依赖：代码之外的运行条件 程序的运行依赖可以分为两类。一类是显式依赖：源代码中直接引用的第三方库、框架和工具，通常记录在依赖清单（如 `go.mod`、`requirements.txt`）中。另一类是隐性依赖：程序并未在代码中声明，但运行时仍然需要的系统条件，例如特定版本

的 glibc、特定路径下的证书文件、特定的内核参数或时区数据。隐性依赖不出现在依赖清单中，往往在更换机器后才暴露。

以第四章的游戏后端为例：大厅服在启用 CGO 的场景下会链接宿主机的系统库，战斗服依赖高精度定时器接口，Redis 和 PostgreSQL 各自有版本兼容性要求，入口层的配置文件引用了特定的模块路径。这些依赖大多不会出现在代码的依赖清单中，正是前面所说的隐性依赖。在一台全新的服务器上重现这套环境，运维人员需要逐一安装依赖、统一版本、调整配置，这些步骤需要在每台服务器上重复一遍。当需要紧急扩容十台服务器时，这种逐台手工操作难以在短时间内完成。

隐性依赖导致的典型故障包括：动态库版本不匹配导致进程启动即崩溃、证书或时区文件缺失导致 TLS 握手失败或时间戳错误、目录权限不同导致写入被拒绝。这些问题的共同特点是：在开发机上不会触发，因为开发机的环境恰好满足了这些隐性依赖；换到另一台机器后条件不再成立，故障才会出现。对在线游戏而言，环境问题的影响集中在扩容和故障恢复两个场景，这两个场景正是要求在最短时间内启动新的服务实例。

三种环境交付方案：手工安装、虚拟机与容器 环境一致性问题有三种主要的解决思路，它们在一致性保证、资源开销和扩容效率之间做出不同取舍。

第一种方案是手工安装依赖。运维人员登录每台服务器，按照文档逐步执行 `apt install`、`pip install`、修改配置文件等操作。这种方式直接且无额外运行开销，但同一份文档由不同的人在不同时间执行，结果往往不同。软件源更新、软件包废弃、操作顺序颠倒都会导致环境偏移，而且这种偏移通常是隐性的，直到线上出现异常时才被发现。

第二种方案是虚拟机镜像。将应用连同完整的 Guest OS、内核和系统库一起封装为虚拟机镜像，部署时直接启动镜像实例。这种方式能够固定虚拟机内部的操作系统、系统库和应用文件，显著减少不同服务器之间的环境差异。但每个虚拟机实例都要运行一套完整的操作系统内核，通常比容器占用更多内存，启动路径也 longer；实际开销取决于镜像、虚拟化平台和硬件配置。当游戏服务需要在短时间内扩容多个实例时，虚拟机的启动延迟会显著拉长扩容响应时间。

第三种方案是容器。容器只打包应用运行所需的用户态环境（代码、运行时、系统工具、依赖库），不包含完整的操作系统内核，而是直接共享宿主机的 Linux 内核。这种方式同样能够固定用户态环境：在处理器架构一致、宿主机内核接口兼容的前提下，同一份容器镜像可以提供相同的代码、工具和依赖库；同时避免了虚拟机的内核冗余开销。因此，容器通常比完整虚拟机更小、启动更快，但实际体积和启动时间仍取决于镜像内容、运行时、硬件和初始化过程。

	手工安装	虚拟机	容器
环境一致性	差	好	好
资源开销	无额外运行层	较高	较低
启动速度	—	通常较慢	通常较快
可重复性	差	好	好
扩容效率	极低	低	高

表 6: 三种环境交付方式的对比

表 6 从五个维度对比了三种方案。手工安装不增加运行开销，但无法保证跨机器的一致性和可重复性。虚拟机提供了完整的环境隔离，但携带了整套操作系统的资源成本。容器只携带用户态环境，共享宿主机内核，在一致性和轻量之间取得了折中，代价是隔离强度弱于虚拟机。

容器是一个通用概念，多个技术实现都遵循上述思路。其中 Docker 在工程中使用最广泛，它将环境交付的单位从源代码或安装包变为完整的运行环境镜像：同一份镜像在开发机和云端服务器上提供相同的用户态条件。Docker 的核心机制围绕三个问题展开：镜像如何分层构建和复用，容器如何在共享内核上实现隔离，数据如何从容器生命周期中分离。

5.1.2 容器的基本原理与使用方法

Docker 将容器技术的构建、分发和运行三个环节标准化为一套工具链。开发者通过 Dockerfile 描述环境构建过程，构建产物是一份不可变的镜像；镜像可以推送到仓库供任意机器拉取；拉取后在任意支持 Docker 的宿主机上以容器形式运行。下面先建立镜像、容器与隔离机制，再把网络、镜像构建、仓库分发和实例运行连成一条完整的部署链路。

镜像与容器：模板与实例 Docker 中有两个必须区分的核心概念：镜像（Image）与容器（Container）。镜像、容器可写层和数据卷具有不同的生命周期，只有先区分这三类对象，才能判断哪些内容可以随实例删除，哪些状态必须长期保留。图 58 将这种生命周期关系置于同一模型中。

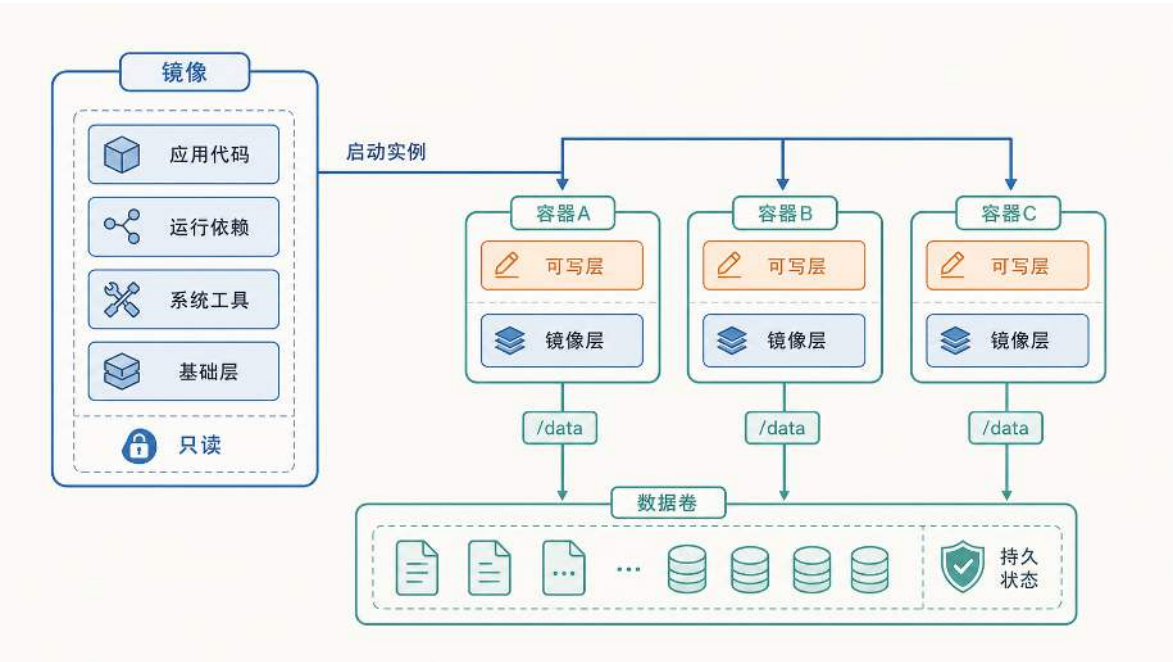


图 58: 镜像、容器与数据卷的生命周期关系：镜像负责复用，容器负责运行，数据卷保存需要长期保留的状态。

镜像由多层只读文件系统叠加而成，从底向上依次是基础层（精简的操作系统用户空间）、系统工具、运行依赖和应用代码。除了文件内容，镜像还包含一组运行元数据：默认启动命令

(ENTRYPOINT/CMD)、工作目录、环境变量默认值和对外声明的端口。镜像一旦构建完成就不再被修改，这一不可变性保证了环境一致性：任意一台机器拉取到同一份镜像后，得到的文件内容和启动方式都是确定的。

当镜像在宿主机上启动时，Docker 在只读镜像层之上挂载一层独立的可写层，形成容器。多个容器可以共享同一份只读镜像模板，各自拥有独立的可写空间、进程空间和网络环境。运行期间程序产生的状态变更（写日志、生成缓存、创建临时文件）都发生在各自的可写层中，镜像层始终保持只读。

可写层的生命周期与容器绑定，容器被删除后，其可写层随之消失，写入其中的数据永久丢失。对于数据库文件、玩家存档等需要长期保留的数据，Docker 提供了数据卷（Volume）机制：数据卷独立于容器生命周期，通过挂载操作映射到容器内的特定路径（如 `/data`）。程序向该路径写入数据时，数据直接落入宿主机磁盘，不经过可写层。这样，计算逻辑（容器）与持久数据（卷）的生命周期被分离：删除容器或升级镜像后，只要挂载同一个数据卷，新容器仍然能接管之前的数据。

镜像不可修改这一性质还带来一个工程问题：同一份镜像要部署到开发、测试、生产等不同环境时，数据库地址、端口号、密钥等配置各不相同，如何处理？做法是将程序本体固化在镜像中，将随环境变化的配置留到运行时注入。注入方式包括环境变量（通过 `-e` 参数或 Compose 的 `environment` 字段传入）、挂载配置文件和挂载数据卷。这样，同一份镜像无需重新构建即可在多种环境中运行，差异完全来自显式声明的运行时配置。这一思路对应 Twelve-Factor App 方法论²⁹中“将配置存储在环境中”这一原则：应用制品本身不包含环境相关信息，环境差异通过运行时注入解决。

至此，本节建立了 Docker 中三个核心对象的关系：镜像提供只读的环境模板，容器在镜像之上添加可写层形成运行实例，数据卷将需要长期保留的状态从容器生命周期中分离出来。镜像不可修改保证了环境一致性，运行时注入解决了跨环境配置差异。在此基础上，一个关键问题尚待回答：容器如何在共享宿主机内核的前提下实现进程和资源的隔离？

隔离原理：Namespace 与 Cgroups 前面提到，容器与虚拟机不同：容器不包含独立的操作系统内核，而是共享宿主机的 Linux 内核。这就带来一个问题：多个容器运行在同一个内核之上，如果没有额外机制，它们就能看到彼此的进程、访问同一套网络接口、争抢同一份 CPU 和内存资源。容器隔离要解决的正是这两件事：让每个容器只能看到属于自己的系统资源视图，以及限制每个容器能消耗的物理资源总量。

虚拟机早已解决了同样的问题，但它的做法是为每个实例配备独立内核，由 Hypervisor 提供硬件级隔离。不同 VM 中的进程运行在完全独立的内核实例上，彼此无法看到对方的进程列表或访问对方的内存空间。这种隔离非常彻底，但每个 VM 都要为 Guest OS 和内核付出数百 MB 到数 GB 的内存开销，传统虚拟机的启动时间通常以分钟计。

容器选择了另一条路径：在共享内核上用软件机制达到类似效果。多个容器各自只包含应用和用户态库，底部共享同一个 Linux 内核。内核通过 Namespace 提供视图隔离，通过 Cgroups

²⁹Twelve-Factor App 的核心主张包括代码与配置分离、依赖显式声明、进程无状态化等。

提供资源配额限制。共享内核降低了运行开销，也使隔离能力依赖内核机制；图 59 将这一取舍与虚拟机的独立内核方案置于相同条件下比较。

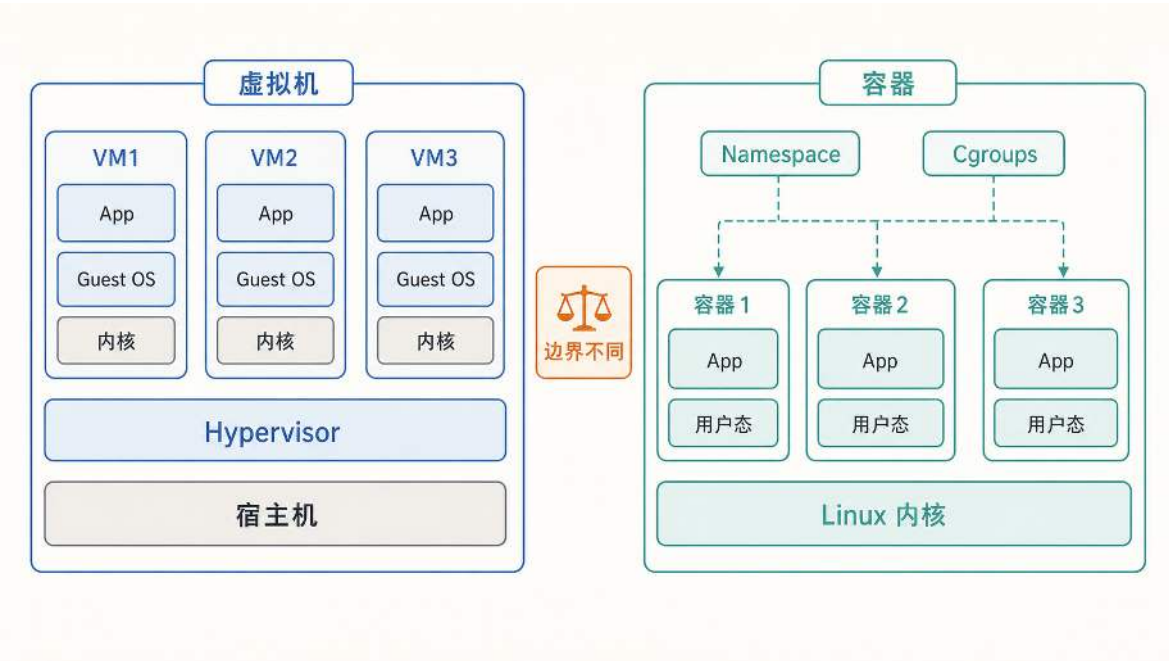


图 59: 虚拟机通过独立 Guest OS 和内核提供强隔离；容器共享 Linux 内核，依靠 Namespace 和 Cgroups 获得更轻量的隔离。

虚拟机用独立内核换取强隔离，容器用共享内核换取轻量和快速启动。

Namespace（命名空间）解决的是“容器能看到什么”的问题。Linux 内核为每个容器创建一组独立的命名空间，使容器只能看到属于自己的系统资源。PID Namespace 让容器内部只能看到自己的进程列表，无法感知宿主机或其他容器中的进程。NET Namespace 让每个容器拥有独立的网络栈（IP 地址、端口空间、路由表），同一个端口号在不同容器中互不冲突。MNT Namespace 让每个容器拥有独立的文件系统挂载视图，容器内的路径结构与宿主机或其他容器不同。通过这些命名空间的组合，容器内的程序“认为”自己运行在一台独立的机器上，而实际上它只是宿主机上一个受限的进程。

Cgroups（控制组）解决的是“容器能用多少”的问题。即使容器通过 Namespace 看不到其他容器的进程，它仍然与其他容器共享同一份物理 CPU 和内存。如果某个容器中的程序出现内存泄漏或死循环，在没有限制的情况下它可能耗尽整台宿主机的资源，导致同一宿主机上的所有其他容器一起受到影响。Cgroups 为每个容器设定 CPU 时间片上限和内存使用上限：容器的资源消耗达到上限后，内核会限制其继续获取 CPU 时间或直接触发 OOM（Out of Memory）终止，而不是让它影响其他容器。

将两个机制合在一起看：Namespace 提供视图隔离（容器看不到别人），Cgroups 提供资源隔离（容器用不了别人的份额）。二者共同使得容器在共享内核的前提下获得了进程、网络、文件系统和资源配额四个维度的独立性。

与虚拟机相比，容器的隔离代价更低（无需额外内核和 Guest OS），但隔离强度也更弱。虚拟机的隔离由独立内核保证，一个 VM 中的内核漏洞不会影响到另一个 VM。容器的隔离由共享

内核中的软件机制保证，如果 Linux 内核本身存在漏洞，理论上一个容器可能突破 Namespace 和 Cgroups 的限制影响其他容器。因此，在对隔离强度要求更高的多租户场景中，工程实践会在容器之外叠加更强的隔离层（如轻量级虚拟机或硬件辅助隔离），在启动速度和安全性之间取得折中。Namespace 的创建方式、Cgroups 控制文件与配额周期等内核实现，将在第 6.1 节进一步展开。

隔离带来了安全性，但也引入了一个新问题：每个容器拥有独立的 Network Namespace，意味着容器之间以及容器与外部客户端之间无法像同一台机器上的进程那样直接通信。第四章中大厅服、战斗服、Redis 和 PostgreSQL 需要互相连接，玩家客户端也需要从外部访问入口层。Docker 通过虚拟网桥和端口映射在网络隔离的前提下解决这两类通信需求。

容器网络：Network Namespace 与端口映射 回顾第四章的部署场景：大厅服需要连接 Redis 读取缓存，需要连接 PostgreSQL 读写玩家数据；入口层需要把玩家请求转发给大厅服；玩家客户端需要从公网连接入口层。在容器化之前，如果这些进程运行在同一台服务器上，大厅服用 127.0.0.1:5432 就能连到本机的 PostgreSQL，用 127.0.0.1:6379 就能连到本机的 Redis。

容器化之后，这种连接方式会失效。上一节介绍的 Network Namespace 为每个容器分配了独立的网络栈：每个容器拥有自己的 IP 地址、网卡和路由表。这意味着容器内的 127.0.0.1 只指向容器自身的网络空间，不再指向宿主机，也不指向同一宿主机上的其他容器。如果大厅服容器中仍然将数据库地址配置为 127.0.0.1:5432，连接目标会变成大厅服容器自己的 5432 端口（那里什么都没有），而不是 PostgreSQL 容器，连接会立即失败。

网络隔离带来了两个需要重新解决的问题：第一，同一宿主机上的多个容器之间如何互相找到对方并建立连接；第二，宿主机外部的玩家客户端如何到达容器内部的服务。Docker 分别用用户自定义网络和端口映射来解决这两个问题。

先看容器间通信。Docker 允许创建用户自定义网络，本质上是宿主机内部的一张虚拟局域网。多个容器接入同一张网络后，每个容器获得一个网络内部 IP（例如 172.18.0.2、172.18.0.3），并通过虚拟网卡在这张局域网内收发数据包。但直接使用 IP 地址连接其他容器并不可靠：容器重启后内部 IP 可能发生变化，把 IP 写死在配置中会导致连接中断。Docker 在每张用户自定义网络中运行一个内置 DNS 服务器，维护“服务名 → 容器 IP”的解析关系。当大厅服容器需要连接 Redis 时，它向 DNS 查询 `redis` 这个服务名，DNS 返回 Redis 容器当前的内部 IP（如 172.18.0.4），大厅服再用这个 IP 建立到 172.18.0.4:6379 的 TCP 连接。即使 Redis 容器因为重启导致 IP 从 172.18.0.4 变为 172.18.0.6，服务名 `redis` 始终有效，调用方的配置不需要改动。因此，容器之间互相访问时，地址统一写成“服务名 + 容器内端口”的形式，例如 `redis:6379`、`postgres:5432`。服务名解析、虚拟网卡之间的数据包转发以及底层的 bridge/NAT 规则属于网络虚拟化的实现细节，将在第 6.2 节展开。

再看外部访问。用户自定义网络中的 IP 和端口只在虚拟局域网内可见，宿主机之外的客户端无法直接访问。玩家的游戏客户端运行在自己的手机或 PC 上，它不知道也无法到达 172.18.0.2 这个内部地址。要让外部流量进入容器，需要在宿主机和容器之间建立端口映射：将宿主机上的某个端口转接到目标容器的端口。例如网关容器在内部监听 8080 端口，启动时配置 `-p 8080:8080`。

这条规则的含义是：当外部客户端连接到云服务器公网 IP:8080 时，Docker 把到达宿主机 8080 端口的流量转接到网关容器的 8080 端口。从玩家视角看，连接的是云服务器的公网地址；从网关容器视角看，有一个连接到达了自己的 8080 端口。后续 Compose 配置中的 `ports`：“8080:8080”以及 Kubernetes 中的 `port/targetPort` 描述的都是这种转接关系。

容器间访问和外部访问处于不同的地址范围，前者依赖服务发现，后者依赖宿主机入口。图 60 将两条路径统一到同一网络模型中。

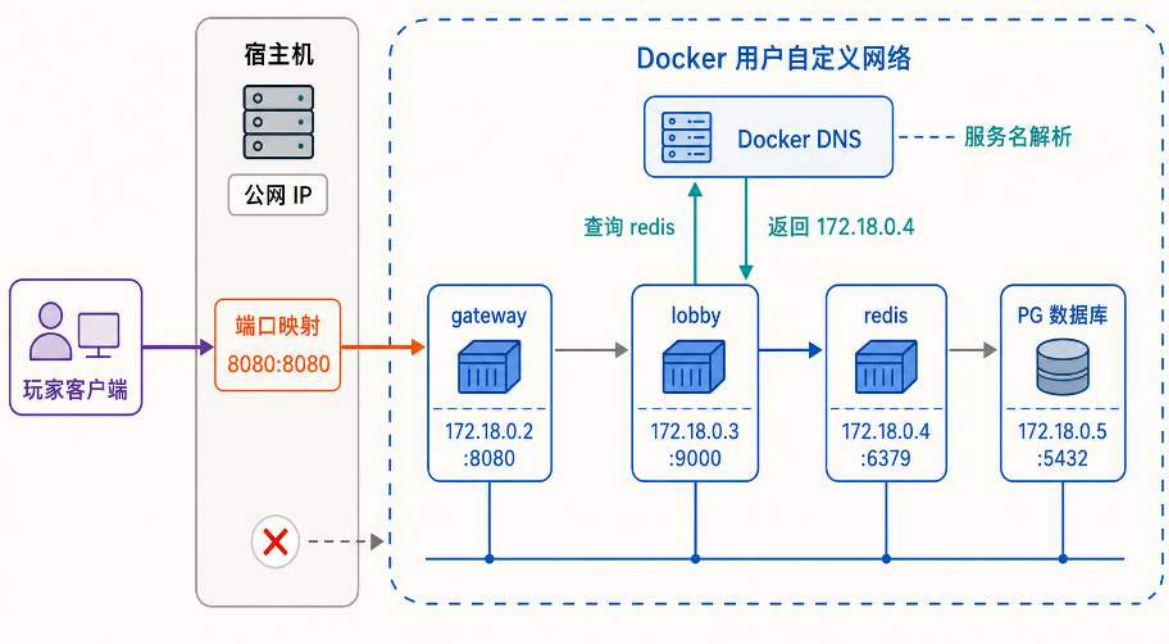


图 60: Docker 用户自定义网络中的两条访问路径：容器间通过服务名和 DNS 在虚拟网络内部互通，外部客户端则必须经过宿主机端口映射才能进入容器。

地址作用域的差异可以进一步归纳为两条配置规则。容器之间互联时，地址写成”服务名:容器内端口”（如 `redis:6379`），由 Docker DNS 负责将服务名解析为容器的内部 IP。外部客户端访问时，地址写成”云服务器公网 IP: 宿主机端口”（如 `203.0.113.10:8080`），由端口映射负责将流量转接到目标容器。两种地址不能混用：容器内部写公网 IP 会绕出虚拟网络再绕回来，外部写服务名则根本无法解析。

至此，Docker 的运行模型已经建立：镜像提供不可变的环境模板，容器在隔离的 Namespace 中运行，数据卷保存持久状态，用户自定义网络和端口映射解决通信问题。但前面的讨论一直假设镜像已经存在，尚未涉及一个基本问题：这份镜像本身是如何构建出来的？

5.1.3 构建、分发与运行的容器化部署闭环

制作镜像：Dockerfile 与分层构建 Docker 通过 Dockerfile 描述镜像的构建过程：开发者在其中逐条写明基础环境、依赖安装和文件复制等步骤，Docker 按顺序执行这些指令，每执行一条就生成镜像中的一个只读层，最终所有层叠加形成完整镜像。

编译型应用如果把工具链和运行环境装入同一镜像，最终制品会携带部署时不再需要的内容。多阶段构建把编译与运行分开，图 61 用这一分离说明镜像为何能够缩小并复用构建缓存。

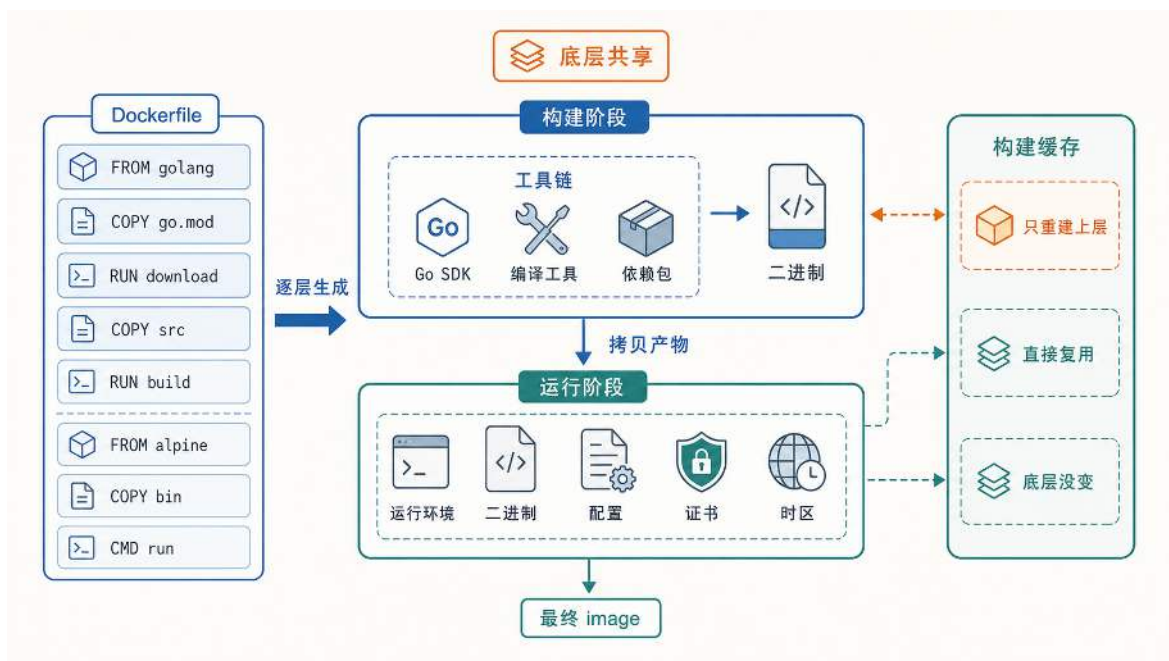


图 61: Dockerfile 的分层构建过程：构建阶段在完整工具链中编译产物，运行阶段只保留最终二进制和最小运行环境，未变化的层直接复用缓存。

构建从 Dockerfile 的第一条指令开始。FROM golang 指定了构建阶段的基础环境，随后 COPY go.mod、RUN download 安装依赖，COPY src、RUN build 编译源码。Go SDK、编译工具和依赖包都存在于这个阶段，最终输出一份二进制文件。接下来 Dockerfile 用第二条 FROM alpine 开启运行阶段，切换到一个仅包含基本用户态工具的极简基础镜像，再把构建阶段产出的二进制、配置文件、证书和时区数据复制进来，形成最终的可分发镜像。

这种两阶段设计的直接效果是镜像体积大幅减小：构建阶段的 Go 工具链（数百 MB）不会出现在最终镜像中，运行阶段只携带实际运行所需的文件（通常十几 MB）。扩容时节点拉取镜像的速度因此加快。

分层构建还带来一个工程上的加速效果：构建缓存。Docker 在构建时逐层检查：如果某一层的指令和输入文件与上次构建完全相同，该层直接复用缓存，不必重新执行。变动频率高的内容（业务源码）放在 Dockerfile 靠后的位置，变动频率低的内容（基础环境和依赖安装）放在靠前的位置。这意味着日常开发中只改动了业务代码时，Docker 只需重建最上面几层，底层全部命中缓存。

当服务数量增加时，为每个服务单独维护一份 Dockerfile 会让构建逻辑的重复随服务数量线性增长。第四章中的游戏后端已经拆分为大厅服与战斗服两个独立模块，后续还可能增加新的服务实例。如果每个模块各有一份几乎相同的 Dockerfile，任何基础环境或依赖安装的改动都需要在多份文件中同步修改，维护成本显著上升。下面这份 Dockerfile 通过构建参数 SERVICE 实现了同一份构建描述适用于多个服务：

通用 Dockerfile：多阶段构建

```
1 FROM golang:1.21-alpine AS builder
2 WORKDIR /app
3 COPY go.mod go.sum ./
4 RUN go mod download
5 COPY . .
6 ARG SERVICE
7 RUN if [ -z "$SERVICE" ]; then echo "ERR: SERVICE not set" ; exit 1; fi
8 RUN CGO_ENABLED=0 GOOS=linux go build -o server_bin ./cmd/${SERVICE}
9 FROM alpine:latest
10 RUN apk --no-cache add ca-certificates tzdata
11 WORKDIR /root/
12 COPY --from=builder /app/server_bin .
13 COPY --from=builder /app/config ./config
14 CMD [ "./server_bin" ]
```

这份 Dockerfile 中有三处值得注意的工程选择，分别对应镜像体积、构建速度和运行时独立性。

第一处是多阶段构建。Dockerfile 中出现了两条 FROM 指令：第一条 FROM golang:1.21-alpine AS builder 引入包含完整 Go 工具链的编译环境，第二条 FROM alpine:latest 切换到极简运行环境。运行阶段通过 COPY --from=builder 只从编译阶段取走最终二进制和配置，Go 编译器本身不进入最终镜像。由此产出的镜像通常只有十几 MB，远小于编译环境的数百 MB。

第二处是依赖下载与源码复制的顺序。Dockerfile 先复制 go.mod 和 go.sum 并执行 go mod download，之后才复制全部源码。由于依赖清单的变动频率远低于业务代码，这种顺序保证了大多数构建中依赖下载层命中缓存，只有源码层需要重建。

第三处是静态编译。构建命令中的 CGO_ENABLED=0 指示 Go 编译器生成不依赖任何动态链接库的静态二进制。这意味着运行阶段的 alpine 镜像中不需要安装额外的 .so 文件，容器可以在同架构的任何 Linux 内核上直接启动，不受宿主机用户态库版本的影响。

分发镜像：Registry 与版本锁定 镜像构建完成后，需要一种机制让多台云服务器都能获取同一份镜像。Docker 的标准做法是引入一个集中式的镜像仓库（Registry）：开发机完成构建后将镜像推送到 Registry，部署时各台云服务器再从 Registry 拉取同一份镜像。由于镜像本身是分层的，拉取时 Docker 只需下载本地尚未存在的层，已有层直接复用，传输量通常远小于镜像的完整体积。分发链路还必须保证制品能够传播到多台机器，并且各机器获取的内容保持一致。图 62 将构建、仓库分发和版本校验连成同一过程，为后文区分 Tag 与 Digest 建立基础。

分发链路解决了“从哪里拉取”的问题，但还有一个更隐蔽的问题：如何保证每台机器拉取到的确实是同一份内容？Docker 的版本标识有两种形式。Tag（如 latest、v1.2）是供人阅读的别名，便于记忆和引用，但它是可变的：同一个 Tag 可以在不同时间被重新指向另一份镜像内

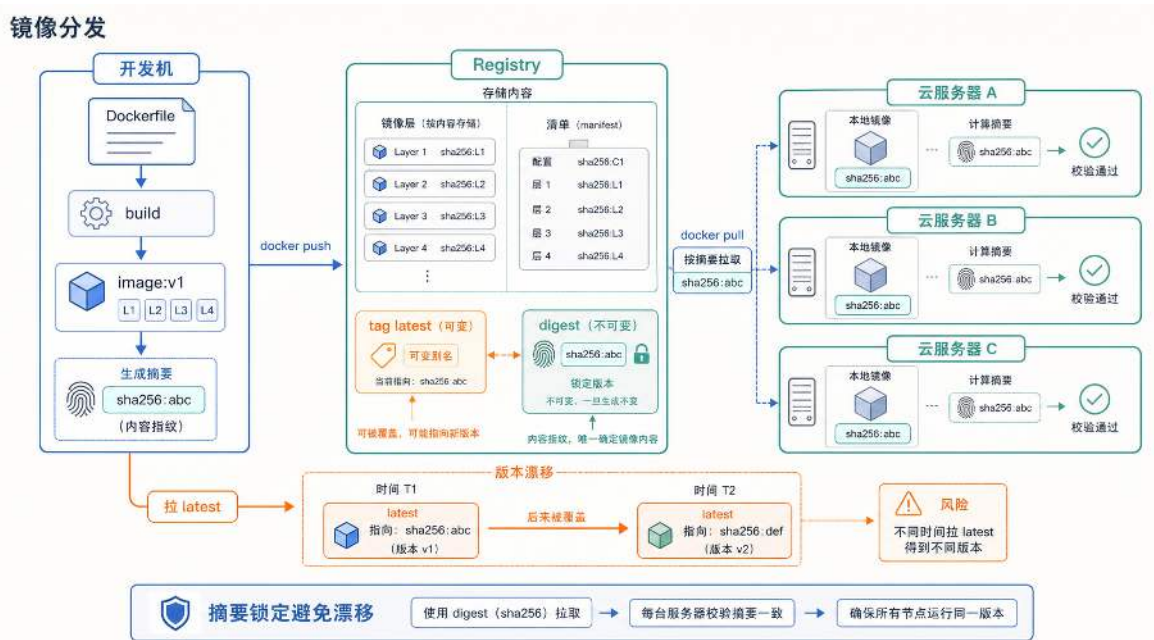


图 62: Docker 镜像的标准化分发链路：开发机推送镜像，云服务器按同一版本摘要拉取，避免线上版本漂移。

容。例如，周一推送的镜像标记为 `v1.2`，周三修复了一个 bug 后重新构建并再次标记为 `v1.2`，此时 Tag 没有变化，但对应的镜像内容已经不同。如果三台云服务器分别在周一、周三和周四拉取 `v1.2`，它们实际运行的镜像可能并不一致，这正是 Tag 漂移问题。

Digest（内容摘要，如 `sha256:a3b7c9...`）是镜像 manifest 的 SHA256 哈希值，在推送到 Registry 时由镜像内容唯一确定。只要两份镜像的 Digest 相同，它们的文件内容就一定相同；内容有任何变化，Digest 必然不同。因此，生产环境中要保证部署的可复现性，部署配置中锁定的应当是 Digest 而非 Tag。写法为 `repo/image@sha256:a3b7c9...`，部署系统按此摘要拉取时，得到的镜像内容是确定的，不受后续重新推送的影响。

构建参数、镜像名和容器服务名这三个标识符名称相近，但分别作用于编译、交付和运行三个不同阶段，初学者容易把它们混为一谈。下面逐一厘清。构建参数（如 `SERVICE=lobby`）对应 Dockerfile 中的 `ARG SERVICE`，决定编译阶段选择哪个入口路径。镜像名与版本（如 `game-lobby:v1`）标识构建产物，是推送到 Registry 和拉取时使用的交付单位。容器服务名（如 `lobby-api`）是容器在 Docker 网络中注册的 DNS 名称，其他容器通过该名称发现并连接它。三者分别回答“编译什么”“交付什么”“在网络中如何被寻址”，各自独立变化：同一份镜像可以在不同环境中以不同服务名运行，同一个服务名也可以随版本升级指向新的镜像。

至此，镜像构建、分层缓存、仓库分发和版本锁定等机制已经逐一介绍完毕。下一步是将它们合在一起使用：把第四章的游戏后端各组件分别装入容器，在一台机器上配置网络、端口和存储，验证整套拓扑能否正常运行。

将游戏后端装入容器 将第四章的游戏后端架构映射到容器：大厅服、战斗服、协调服务器属于计算进程，各自封装为独立镜像；Redis 与 PostgreSQL 属于基础设施进程，同样各自对应一

份镜像；入口层（自研网关或 Nginx 等反向代理）也是一个可容器化的进程。每个容器化组件需要回答两个基本问题：它在容器内监听哪个端口？它要主动连接哪些上游服务的地址和端口？

下面在一台机器上把这些组件以容器形式启动并互相连通，目的是验证前几节介绍的网络、端口、数据卷和配置注入机制在实际部署中如何配合。为简化演示，这里只保留入口层、大厅服、Redis 和 PostgreSQL 四个组件；战斗服与协调服务器的接入方式与大厅服相同，不再重复。

单机部署示例（结构示意）

```
1 docker network create game-net || true
2
3 docker run -d --name postgres --network game-net \
4   -v pg-data:/var/lib/postgresql/data postgres:15
5 docker run -d --name redis --network game-net redis:6
6
7 docker run -d --name lobby-api --network game-net \
8   -e DB_ADDR=postgres:5432 -e REDIS_ADDR=redis:6379 \
9   game-lobby:v1
10 docker run -d --name gateway --network game-net \
11   -p 8080:8080 -e LOBBY_ADDR=lobby-api:8080 \
12   game-gateway:v1
```

这段命令中体现了前几节介绍的五个配置机制。

第一，所有容器通过 `--network game-net` 接入同一张用户自定义网络。接入后，每个容器的 `--name`（如 `postgres`、`redis`、`lobby-api`）自动注册为 Docker 内置 DNS 中的服务名。大厅服连接数据库时使用 `postgres:5432` 而非 IP 地址，正是依赖这一解析机制。

第二，运行时配置通过环境变量注入。大厅服的 `-e DB_ADDR=postgres:5432` 和 `-e REDIS_ADDR=redis:6379` 将上游地址以环境变量形式传入容器，镜像本身不包含任何环境相关的硬编码地址。这是前文介绍的 Twelve-Factor 原则在实际部署中的体现。

第三，端口映射仅出现在需要对外暴露的入口层。只有网关容器使用了 `-p 8080:8080`，将宿主机的 8080 端口转接到容器内部的 8080 端口，供外部玩家客户端连接。其余容器不需要端口映射，因为它们之间的通信完全在 Docker 网络内部通过服务名完成。

第四，PostgreSQL 容器挂载了数据卷 `pg-data`，使数据库文件持久化到宿主机磁盘。即使 PostgreSQL 容器被删除或升级镜像后重新创建，只要挂载同一个数据卷，数据仍然完整保留。

以上五项配置（网络归属、服务名注册、环境变量注入、端口映射和数据卷挂载）是容器启动时需要显式指定的部署参数。但即使只有四个组件，维护这些配置已经暴露出明显的脆弱性：启动顺序完全依赖执行者的记忆（PostgreSQL 和 Redis 必须先于大厅服启动），配置参数散落在各条命令的 `-e` 和 `-p` 标志中无法统一审查，新增一个组件需要手动补齐网络接入、环境变量和启动顺序。当组件数量从四个增长到十几个时，一串 `docker run` 命令将难以维护和复现。

解决这个问题的思路是：把所有组件的镜像版本、网络归属、端口映射、环境变量和数据卷声明集中写入一份文件，用工具统一解析和执行。Docker Compose 正是这种声明式部署方式的

标准实现。

5.2 编排：云上多服务互连与调度的基础

上一部分已经能把单个服务装入容器，但一组在线服务还需要共同启动、互相发现、按约束放置，并在配置变化后自动将实际状态拉回与声明一致。编排解决的就是这组问题：用户描述服务需要达到的状态，平台负责创建实例、连接网络、维护副本，并在现实状态偏离目标时执行纠正。本部分先用 Docker Compose 说明单机多容器编排，再分析跨机器时新增的网络、发现和集中控制问题，最后进入 Kubernetes 的集群级自动化编排。

5.2.1 编排是什么：从手工命令到声明式拓扑

手工管理关注的是“下一条命令执行什么”，编排关注的是“整组服务最终应处于什么状态”。一份编排声明至少需要回答四个问题：运行哪些服务、每个服务需要多少实例、服务之间如何互联、实例失败或配置变化后如何恢复到目标状态。单机工具只能完成其中一部分，集群级平台还要增加跨节点调度、故障补位和滚动更新等能力。

手工管理一组容器时，运维人员需要逐条执行 `docker run` 命令，按记忆维护启动顺序，手动检查每个容器是否存活，出现故障时逐个登录服务器排查。编排则把同样的信息写成一份声明文件：服务需要达到什么状态由文件描述，状态偏离目标时由平台自动纠偏。这种差异不仅体现在工具层面，更体现在管理视角上。手工管理看到的是“这个容器挂了”，编排看到的是“这个服务应当保持 3 个副本，当前只有 2 个，需要补 1 个”。

5.2.2 单机多容器编排：Docker Compose

Compose 文件使用 YAML 格式描述一组容器的期望状态。文件的核心结构是 `services` 块：其中每一个条目定义一类服务，Compose 根据该定义创建一个或多个容器；条目名称同时作为项目网络中的服务名。每个条目内部声明这个容器需要的全部配置：使用什么镜像、注入哪些环境变量、暴露哪些端口、挂载什么数据卷、依赖哪些其他服务。换言之，第 5.1.3 节部署示例中散落在各条 `docker run` 命令里的信息，现在按容器分组写进了同一份文件。

下面这份配置描述了与该示例相同的四个组件（PostgreSQL、Redis、大厅服务和网关）：

Compose 配置示例 (docker-compose.yml)

```
1 services:
2   postgres:
3     image: postgres:15
4     volumes:
5       - pg-data:/var/lib/postgresql/data
6   redis:
7     image: redis:6
8   lobby-api:
9     image: game-lobby:v1
10    depends_on: [postgres, redis]
11    environment:
12      DB_ADDR: postgres:5432
13      REDIS_ADDR: redis:6379
14  gateway:
15    image: game-gateway:v1
16    ports: ["8080:8080"]
17    depends_on: [lobby-api]
18    environment:
19      LOBBY_ADDR: lobby-api:8080
20  volumes:
21    pg-data:
```

执行命令 `docker compose up` 后，Compose 将这份声明转化为实际运行的容器拓扑。同一份声明只有在服务、网络、数据卷、配置和入口规则被一致地转化为运行对象时，才能复现相同的应用拓扑。图 63 将这种从声明到运行状态的映射固定下来。

Compose 的构建过程按以下顺序推进。首先，Compose 创建一张项目专属的用户自定义网络，所有容器将接入这张网络并获得内部 IP。YAML 中不需要显式声明网络名称，Compose 默认整个项目创建一张共享网络。接着，Compose 根据 `depends_on` 确定容器的启动顺序。上例中 `lobby-api` 声明依赖 `postgres` 和 `redis`，`gateway` 又声明依赖 `lobby-api`，因此启动顺序按依赖链推进：先启动数据库与缓存，再启动大厅服，最后启动网关。上例使用的是 `depends_on` 的简写形式，它控制容器进程的启动顺序，但不保证容器内的服务已经就绪。Compose 也支持将依赖条件设置为 `service_healthy`，等待依赖服务通过健康检查后再启动下游；不过健康检查只能覆盖已经声明的检测条件，应用仍应能够处理依赖短暂不可用。例如 PostgreSQL 容器启动后，数据库进程可能仍在执行初始化，尚未开始监听连接；如果 `lobby-api` 此时尝试连接，仍然会失败。因此应用代码中需要包含带退避的连接重试逻辑：每次连接失败后等待一段时间再重试，且等待时间应随失败次数增加而逐渐拉长（这种策略称为退避，backoff）。不能把启动顺序当作依赖在整个运行期间始终可用的保证。每个容器启动时，Compose 将 YAML 中声明的环境

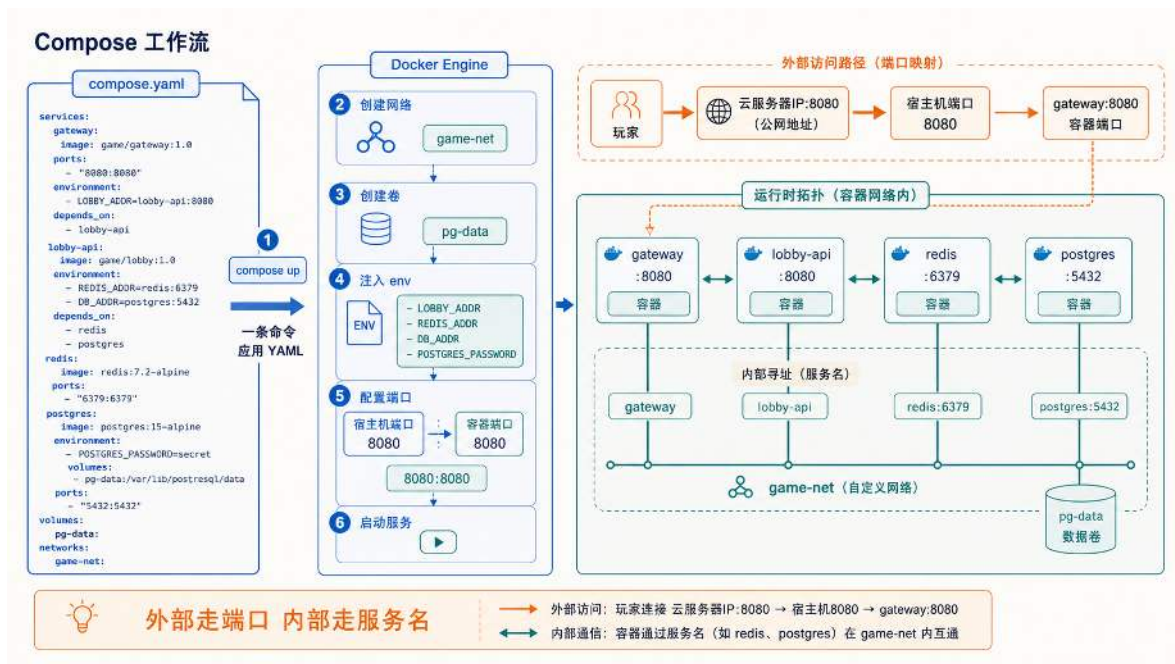


图 63: Compose 将 YAML 声明转化为运行时拓扑：服务、网络、数据卷与环境变量经过构建流程，生成实际运行的容器组及其网络连接。

变量注入容器进程，将 `ports` 字段指定的宿主机端口映射到容器端口，并将 `volumes` 字段指定的数据卷挂载到容器内路径。同时，Compose 将每个服务名注册到网络内置 DNS 中，服务名即为容器间互相访问时使用的 DNS 名称。启动完成后，`lobby-api` 容器访问 `postgres:5432` 时，DNS 将 `postgres` 解析为 PostgreSQL 容器的当前 IP，连接在网络内部直接建立。如果启动过程中出现端口冲突或镜像拉取失败，Compose 会报告具体错误，但已经创建或启动的容器是否保留取决于失败发生的阶段和执行参数，不能把这一过程理解为自动事务回滚。需要恢复到干净状态时，应显式检查项目状态并执行停止或清理操作。

Compose 的另一个重要特性是重复执行时的幂等行为。对同一份 YAML 文件多次执行 `docker compose up`，Compose 会检查当前运行状态与文件描述之间的差异：已存在且配置未变的容器保持运行不做重建，配置发生变化的容器才会被停止并重新创建。这意味着修改 YAML 中某个服务的镜像版本后重新执行，只有该服务会被更新，其余服务不受影响。

Compose 还支持为容器配置重启策略（如 `restart: unless-stopped`）。设置后，容器进程异常退出时 Docker Engine 会自动将其重新拉起，无需人工介入。但这种自动恢复有两个盲区。第一，容器进程可能处于“假死”状态，即进程仍在运行但不再响应请求，`restart` 策略不会触发重启，因为 Docker Engine 只能检测到进程退出，无法判断应用是否还能正常提供服务。要覆盖这类故障，需要应用层面的健康检查机制，这也是集群级编排平台与单机工具的重要差别之一。第二，这种恢复仅限于宿主机本身正常运行的前提下。如果宿主机断电或内核崩溃，运行在其上的所有容器都会一同丢失，Compose 没有能力在另一台机器上补出替代实例。

表 7 从操作模式、管理视角和适用规模等维度比较手工管理与集群级编排。Docker Compose 已具备声明式配置、内置 DNS 服务发现和自动重启等单机能力，但跨机调度、集中配置管理和声明式存储需要集群级平台。编排的核心价值不在于增加几个自动化命令，而在于把管理视角

从”逐个容器”提升到”应用整体”：运维人员描述服务应当达到的状态，平台负责在现实偏离目标时自动纠偏。

表 7: 手工管理与多容器编排的对比

维度	手工管理（如 Shell + docker run）	集群级编排（以 Kubernetes 为例）
操作模式	命令式：按顺序执行命令，每一步依赖前一步成功	声明式：描述期望状态，平台自动收敛现实
管理视角	容器个体：逐个检查每个容器的存活和配置	应用整体：关注服务副本数、互联关系和健康状态
服务发现	需人工配置 IP 和端口，容器重启后重新对齐	内置 DNS 与服务名解析，容器漂移后自动更新
故障处理	人工登录排查、手动重启，响应依赖值班	自动检测失败容器并重新创建，跨机故障时可自动迁移到其他节点
扩缩容	手动执行多次 docker run，逐个修改负载均衡	修改副本数字段即可触发自动调度与扩展
配置管理	环境变量写死在命令行，分散在各条启动脚本中	ConfigMap / Secret 集中管理，与镜像解耦
存储管理	人工指定宿主机路径，迁移时数据跟随困难	声明式卷 (Persistent Volume)，平台负责挂载与迁移
适用规模	单机、开发测试、简单应用	生产环境、大规模集群、微服务架构

5.2.3 从单机到多机：跨机网络、服务发现与集中控制

上一节末尾提到，Compose 配置的重启策略只能在宿主机本身正常的前提下生效。这暴露了 Compose 的根本约束：它的控制对象是单台宿主机上的 Docker Engine。Compose 读取 YAML 后，将配置翻译为一组 API 调用，发送给本机的 Docker Engine；一个 Engine 对应一台机器，因此 Compose 只能将容器编排在一台机器上。

当游戏后端从开发环境走向线上时，单机部署的局限会迅速显现。第一，单机故障等于整体故障：宿主机断电后所有容器丢失，无法在另一台机器上自动补出替代实例。第二，单机资源存在上限：当 CPU 和内存耗尽时，Compose 没有能力将新副本调度到其他机器。第三，跨机器的服务发现和流量入口需要额外维护：Compose 创建的用户自定义网络和 DNS 仅在本机可见，不同机器上的容器无法通过服务名互相访问。

这三项限制分别要求集群提供统一控制面、跨节点网络与服务发现，以及在节点故障后重新放置副本的能力。决定性的变化是期望状态不再绑定到一台 Docker Engine：集群控制面可以在存活节点上恢复缺失副本。图 64 将这一变化置于单机与多机两种管理范围中比较。

统一控制、跨机互联和故障补位构成了多机编排平台的共同问题。下面先用 Docker Swarm 作为过渡实例说明基本机制，再进入功能更完整的 Kubernetes。

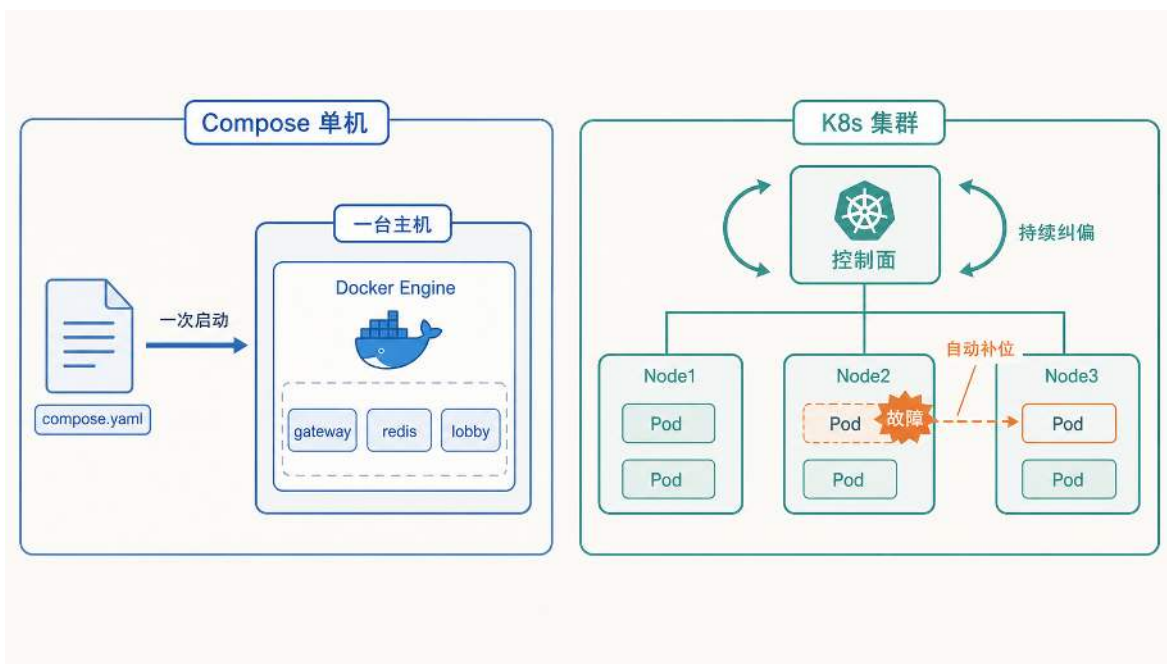


图 64: 单机 Compose 把声明交给一台宿主机；多机集群由控制面持续纠偏，并在节点故障后于存活节点补位。

过渡实例：Docker Swarm Docker 自身为多机场景提供了一套轻量级集群方案 Swarm Mode：将多台机器的 Docker Engine 组织为一个集群，Manager 节点负责控制，Worker 节点负责执行。Swarm 解决了 Compose 无法覆盖的三个关键问题：通过 overlay 网络实现跨节点容器通信，通过集群 DNS 实现跨机服务发现，以及通过 Manager 的统一控制面实现副本分配与故障补位。节点故障后，Manager 依据期望副本数在存活节点上补位，说明统一控制面如何把故障转化为需要纠正的状态偏差。图 65 用正常运行与故障恢复两个阶段验证这一机制。

但 Swarm 的调度规则相对简单，缺少细粒度的亲和性、反亲和性和可扩展控制接口。当集群规模继续增长时，Kubernetes 提供了更完整的机制组合。下面不再比较产品差异，而是直接分析 Kubernetes 如何用同样的”期望状态与实际状态对齐”思路，把控制职责进一步拆分。

5.2.4 Kubernetes 对象、调度与控制面

Kubernetes（简称 K8s）沿用 Swarm 已经建立的”期望状态与实际状态对齐”思路，并进一步把对象定义、放置决策、控制职责和状态调谐拆分为独立机制。下面先建立三个基本对象及其协作关系，再看调度器如何决定副本放在哪台 Node 上，最后分析控制面各组件怎样共同维持集群状态。

Pod、Deployment 与 Service 的协作关系 Pod、Deployment 与 Service 分别承担运行、维护和访问职责，三者不通过固定实例绑定。Deployment 依据标签维持一组可替换的 Pod，Service 通过 Selector（如 `app=lobby`）选择其中已经就绪的实例，因此 Pod 被重建后仍可重新加入同一服务。图 66 建立了这三个对象的协作关系，下面依次说明各对象如何参与部署与恢复。

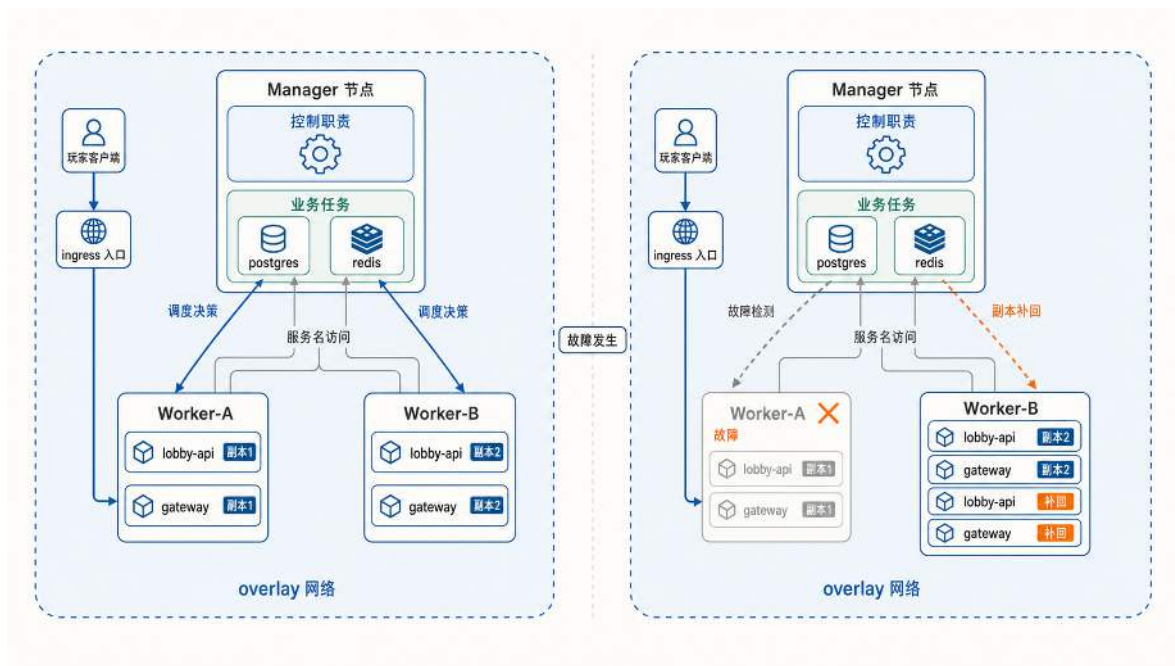


图 65: Docker Swarm 集群的正常运行与故障恢复: Manager 负责控制与调度, Worker 执行业务任务; 节点故障后 Manager 在存活节点上自动补回缺失副本。

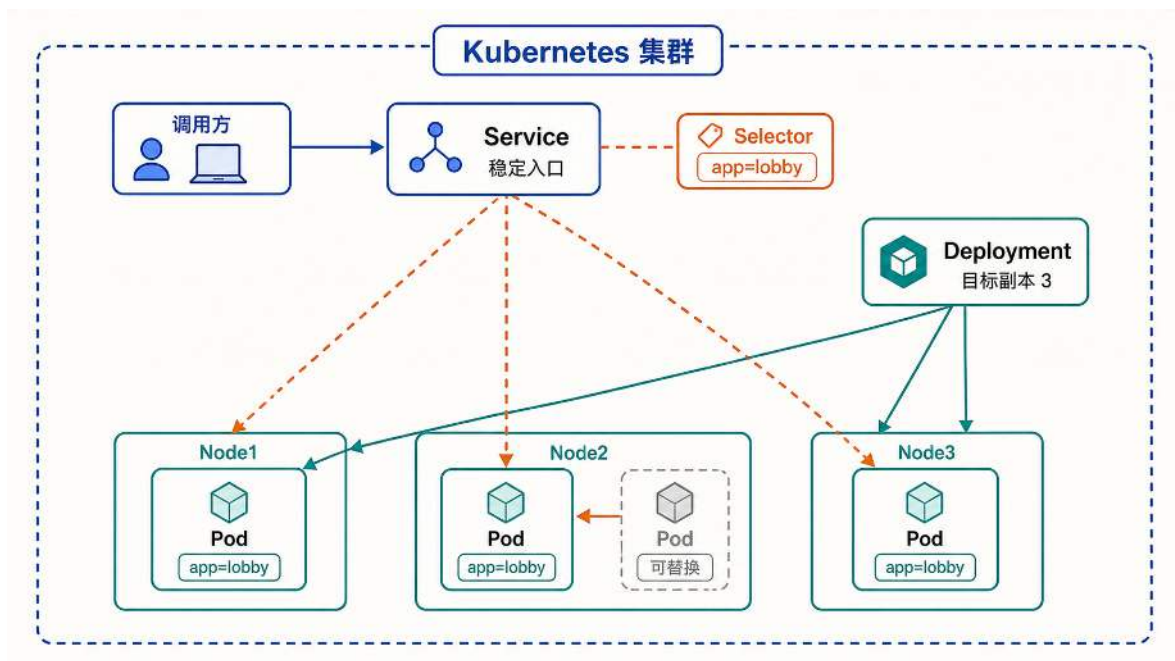


图 66: K8s 核心对象总览: 调用方通过 Service 的稳定入口访问服务, Service 通过 Selector 选择符合标签的 Pod, Deployment 维持 Pod 的副本数目标, Pod 分布在多个 Node 上且可被替换。

Pod：最小的调度单位 在 K8s 的扩容流程中，Scheduler 选择节点后，kubelet 在本地启动的对象是 Pod 而非单个容器。Pod 是 K8s 中最小的调度和管理单位，它包含一个或多个共享网络命名空间和存储卷的容器。

为什么不直接以容器为最小单位？因为有些场景下，两个进程必须紧密协作：它们需要通过 localhost 互相通信，或共享同一份临时文件。例如游戏的战斗服容器旁边可能需要一个日志采集容器（通常称为 sidecar），两者共享同一个日志目录，战斗服写入日志，sidecar 负责将日志转发到集中存储。如果把它们放在不同的调度单位中，就可能出现战斗服已经启动但日志采集还未就绪的不一致状态。

Pod 将这类强耦合关系固化为平台可以理解的内容：同一个 Pod 内的容器被调度到同一个节点，共享网络和存储。容器进程异常时，kubelet 可以在原 Pod 内单独重启容器；Pod 被删除或所在节点失效后，控制器会创建一个新的 Pod，新 Pod 可能被调度到另一节点。这个过程是“创建替代 Pod”，而不是把原 Pod 迁移过去。对于大多数无状态服务（如 lobby-api、gateway），一个 Pod 内通常只有一个业务容器，此时 Pod 可以简单理解为“一个容器实例加上它的网络身份和生命周期”。

Pod 本身是短暂的。它可能因为节点故障被销毁，也可能在滚动更新中被新版本替换。K8s 不试图让某个具体的 Pod 永远存活，而是通过更高层的机制保证“始终有足够数量的同类 Pod 在运行”。这个机制就是 Deployment。

Deployment：声明副本数与更新策略 以一个典型的扩容场景为例：Controller Manager 检测到“期望副本数为 4 而实际为 2”后发起纠正。这里的“期望副本数”就记录在 Deployment 对象中。Deployment 是用户向 K8s 表达以下意图的方式：这个服务应该始终保持多少个 Pod 副本在运行，当需要更新版本时应该按什么策略进行替换。

以游戏的大厅服为例，一份 Deployment 的关键字段如下：

大厅服 Deployment 配置示例

```
1  apiVersion: apps/v1
2  kind: Deployment
3  metadata:
4    name: lobby-api
5  spec:
6    replicas: 3
7    selector:
8      matchLabels:
9        app: lobby-api
10   strategy:
11     type: RollingUpdate
12     rollingUpdate:
13       maxSurge: 1
14       maxUnavailable: 0
15   template:
16     metadata:
17       labels:
18         app: lobby-api
19     spec:
20       containers:
21       - name: lobby
22         image: game-lobby:v2
23         readinessProbe:
24           httpGet:
25             path: /healthz
26             port: 9000
27           periodSeconds: 5
```

这段配置表达了三层意图。第一层是副本数：**replicas: 3** 告诉平台大厅服应长期保持 3 个目标 Pod。目标数量不等于可用数量：Pod 只有在成功调度、容器启动并通过就绪检查后，才算作可以接收流量的就绪副本。当某个 Pod 因节点故障消失时，Controller Manager 中的 Deployment 控制器会检测到实际数量低于期望值，并创建新的 Pod 来补位。

第二层是更新策略。**strategy** 字段定义了版本升级的节奏：**maxSurge: 1** 表示更新过程中最多比目标多出 1 个 Pod（即先启动新的，再下线旧的），**maxUnavailable: 0** 表示控制器以“不主动减少现有可用副本”为更新约束。只有在集群仍有可调度容量、新 Pod 能成功启动并通过就绪检查时，更新过程中才能维持不少于 3 个就绪副本；否则滚动更新会停滞，而不是凭配置自动获得可用容量。这种逐步替换而非一次性全换的方式就是滚动更新（Rolling Update）。

第三层是就绪判定。**readinessProbe** 告诉 kubelet 如何判断一个新启动的 Pod 是否已经准

备好接收流量。在上面的配置中，kubelet 每 5 秒向 Pod 的 9000 端口发送一次 HTTP 请求，只有收到成功响应后，该 Pod 才会被加入 Service 的后端列表。这解决了一个实际问题：容器进程启动成功不代表服务就绪，大厅服可能还在加载配置、建立数据库连接或预热缓存，如果此时就接入玩家流量，玩家会看到超时或错误。

对于游戏场景中的长连接服务（如 gateway 维护的 WebSocket 连接），滚动更新还涉及”排水”问题：旧 Pod 在下线前需要等待已有连接自然结束或主动迁移，而不是直接切断。Readiness Probe 能让平台识别”这个 Pod 是否适合接入新流量”，但存量连接何时结束取决于业务逻辑。K8s 提供了 preStop 钩子和 terminationGracePeriodSeconds 来给旧 Pod 留出排水时间，具体的优雅关闭策略需要应用代码配合实现。

与 readinessProbe 不同，livenessProbe 用于检测已经运行的 Pod 是否还活着；如果 livenessProbe 连续失败，kubelet 会重启该 Pod 内的容器，从而覆盖”进程假死但仍在运行”的盲区。

Service：稳定的访问入口 前面讨论容器网络时，Docker 的内置 DNS 解决了单机上的服务名解析问题：容器通过服务名找到对方的 IP。在集群环境中，原 Pod 消失后，控制器创建的替代 Pod 可能被调度到另一节点；新 Pod 的 IP 地址也可能变化。如果调用方将某个 Pod 的 IP 写死在配置中，Pod 一旦被替换，连接就会中断。

Service 解决的正是这个问题。它为了一组 Pod 提供一个稳定的访问入口（一个固定的集群内 IP 和 DNS 名称），调用方只需访问 Service 名称（如 lobby-api），平台负责将流量转发到当前存活且就绪的 Pod。

Service 如何知道应该把流量转给哪些 Pod？答案是标签（Label）与选择器（Selector）。在上面的 Deployment 配置中，每个 Pod 都带有 app: lobby-api 标签。Service 的定义则通过选择器指定它关心的标签：

大厅服 Service 配置示例

```
1  apiVersion: v1
2  kind: Service
3  metadata:
4    name: lobby-api
5  spec:
6    selector:
7      app: lobby-api
8    ports:
9      - port: 9000
10      targetPort: 9000
```

这段配置的含义是：凡是带有 app: lobby-api 标签且通过 Readiness 检查的 Pod，都会被自动加入这个 Service 的后端列表。当 Pod 因故障被销毁、新 Pod 被补位创建后，只要新 Pod 也带有相同标签并通过就绪检查，它会自动出现在 Service 的后端列表中，无需人工修改任何配

置。

这三个对象由标签和状态连接起来：调用方访问 Service，Service 通过 Selector 找到符合标签且已经就绪的 Pod，Deployment 持续保证这些 Pod 的数量和版本达标。当某个 Node 故障时，Deployment 在其他 Node 上补出新 Pod，新 Pod 通过就绪检查后被 Service 纳入后端列表。整个恢复过程中，调用方使用的入口地址始终不变。

Service 解决了”调用方访问谁”的问题，但还没有解释”不同 Node 上的 Pod 如何互相信信”。K8s 的网络模型要求每个 Pod 拥有一个集群内唯一的 IP，且不同 Node 上的 Pod 能够直接互通。这部分底层机制（地址分配、跨节点路由、VXLAN 封装）由 CNI（Container Network Interface）插件承担，将在第 6.2 节结合网络虚拟化展开。

三个抽象各有分工：Pod 定义运行单元的边界，Deployment 定义副本数量与更新策略，Service 定义稳定的访问入口。对于本书的游戏后端，gateway、lobby-api、coordinator、battle 都可以按这个模式部署：每个组件对应一个 Deployment 和一个 Service，每个副本是一个 Pod，服务间通过 Service 名称互相访问。至于 redis 与 postgres 这类有状态组件，它们同样需要稳定入口，但对”可替换性”有额外限制（数据不能随 Pod 消失），这个取舍会在后续章节讨论。

实际部署中还需要解决跨环境配置差异：同一份镜像要在开发、测试、生产三个环境中运行，数据库地址和密码各不相同。K8s 用 ConfigMap 保存非敏感配置（服务地址、日志级别、超时时间），用 Secret 保存敏感配置（数据库口令、证书私钥），二者以环境变量或文件挂载的方式注入 Pod。这样镜像本身不必为不同环境重复构建，环境差异完全由外部配置对象承载。

上述对象已经覆盖了服务部署的核心需求：多节点分布、副本数量保障和稳定入口。ConfigMap 和 Secret 负责将环境配置与镜像解耦，可以独立创建后再注入 Pod。将这些对象组织成一条部署链路，可以进一步说明从集群资源准备到服务具备访问能力，各类对象依次承担什么职责。

部署链路：从空集群到可访问的服务 第一步，准备集群资源。控制面负责保存和协调集群状态，工作节点负责承载 Pod。教学环境可以使用单个控制面节点搭建最小集群；生产环境通常采用高可用控制面，或使用云平台托管的控制面，避免控制节点成为单点故障。工作节点注册后，平台可以通过统一的状态视图管理整组计算资源。

第二步，声明 Deployment。运维人员编写一份 YAML 文件，指明服务需要运行几个 Pod 副本、使用哪个镜像、每个副本请求多少 CPU 和内存。将这份文件提交给 API Server 后，Controller Manager 会检测到实际副本数与期望值的差距，并推动创建新的 Pod。

第三步，声明 Service。由于 Pod 的 IP 可能随重建而变化，直接访问 Pod 不可靠。运维人员再编写一份 Service YAML，通过标签选择器把一组 Pod 绑定到一个固定的集群内 DNS 名称和虚拟 IP 上。集群内部的其他服务通过这个 DNS 名称互相访问，不受 Pod 重建的影响。

第四步，提供受控的外部入口。集群内的 Service 默认只在内部网络可达。生产环境通常在集群外设置统一入口，由它承接公网地址、TLS 终止和请求路由，再把流量导向集群内的 Service；常见实现包括云负载均衡、Ingress 或 Gateway。NodePort 直接在节点上暴露端口，更适合教学实验或作为外部入口的底层接入方式，不应让客户端长期依赖某个工作节点的公网地址。无论采

用哪种实现，外部流量都应先经过稳定入口和 Service，再到达当前就绪的 Pod。

这四步完成后，服务已经具备基本的可访问性。但发布到一半突然出错、节点宕机、镜像拉取失败时，系统到底处于什么状态，又靠什么机制回到正常？这正是声明式模型与调谐循环要解决的问题。

5.2.5 资源调度：放置决策与资源约束

前面讨论的副本故障补位、版本滚动更新和启动就绪检查三种场景，都涉及控制器创建新 Pod。在前面的分析中，这些新 Pod 默认被放在“可用 Node 上”启动。但集群通常有多台 Node，资源余量各不相同，已有的负载分布也不同。如果放置不加控制，两类问题会传导到玩家侧：一是控制器已经决定补副本，却没有任何一台 Node 的剩余资源能承载这个 Pod，Pod 停留在待调度状态，玩家看到容量始终不恢复；二是 Pod 被放到已经承载了大量同类副本的 Node 上，该 Node 一旦故障，对局大面积中断。

K8s 通过调度器 (Scheduler) 解决这一决策。控制器负责决定“做什么” (补副本、替换版本)，调度器负责决定“放在哪台 Node 上”。每个新创建的 Pod 先进入调度队列，调度器根据 Pod 声明的资源需求和集群各 Node 的当前状态，选定一台目标 Node 并完成绑定。调度器的决策质量取决于两个前提：Pod 是否准确声明了自己的资源需求，以及调度器能否获取各 Node 的实时可分配资源。下面先讨论第一个前提。

资源声明：requests 与 limits 调度器判断节点是否能够承载一个 Pod，主要依据 Pod 对资源需求的显式声明。如果 Pod 没有声明资源需求，调度器就无法按照其真实消耗预留容量，可能把过多工作负载放到同一节点。K8s 用两个字段承载这一声明：**requests** 和 **limits**。

requests 表示 Pod 在调度时申请预留的资源量 (CPU 和内存)。调度器在选择 Node 时，会计算该 Node 上所有已有 Pod 的 **requests** 总和，只有剩余可分配量大于等于当前 Pod 的 **requests**，该 Node 才进入候选。**limits** 表示 Pod 运行时允许使用的资源上限。它不参与调度决策，而是在运行时由 Cgroups 执行：CPU 超出 **limits** 时进程被限速 (throttle)，内存超出 **limits** 时进程被终止 (OOM Kill)。

两个字段的分工可以这样理解：**requests** 决定 Pod 能否被放上某台 Node (调度阶段)，**limits** 决定 Pod 放上去之后能消耗多少资源 (运行阶段)。如果 **requests** 设置得过低，调度器会低估 Pod 的真实消耗，多个 Pod 被叠加到同一台 Node 上，运行时出现资源争用；如果 **limits** 设置得过紧，Pod 在负载升高时频繁被限速或终止。前面隔离一节介绍的 Cgroups 正是 **limits** 的底层执行机制：**limits** 字段的值最终转化为 Cgroups 的 CPU 时间片上限和内存上限配置。

过滤与打分：调度器的两步决策 当一个新 Pod 被创建并进入调度队列后，调度器按两步完成放置。

第一步是过滤 (Filtering)。调度器逐一检查集群中的每台 Node，排除不满足条件的 Node。过滤条件包括：该 Node 的剩余可分配资源是否大于等于 Pod 的 **requests**；该 Node 是否被标记为不可调度 (例如正在维护)；Pod 是否声明了特殊约束 (例如必须运行在配备 GPU 的 Node

上)。过滤结束后，剩余的 Node 构成候选集合。如果候选集合为空，Pod 停留在 Pending 状态，直到有 Node 释放出足够资源。

第二步是打分 (Scoring)。调度器对候选集合中的每台 Node 评分，选择得分最高的一台作为目标。得分由启用的评分插件共同决定，可以考虑资源利用与均衡、亲和或反亲和规则、拓扑分布等因素，并不存在对所有集群都成立的单一“剩余资源越多越优”规则。

`battle` 服务的一次补位调度可以把上述过程具体化。待调度 Pod 声明了 `requests: {cpu: 2, memory: 4Gi}`，集群中有三台 Node；调度器先过滤掉资源不足或不满足约束的 Node。本例假设评分策略同时偏好保留资源余量和分散同类副本，因此 Node-C 得分最高，Pod 最终绑定到 Node-C。这次放置结果说明，调度由硬约束过滤和软目标排序共同产生；图 67 用一次完整决策验证这一点。

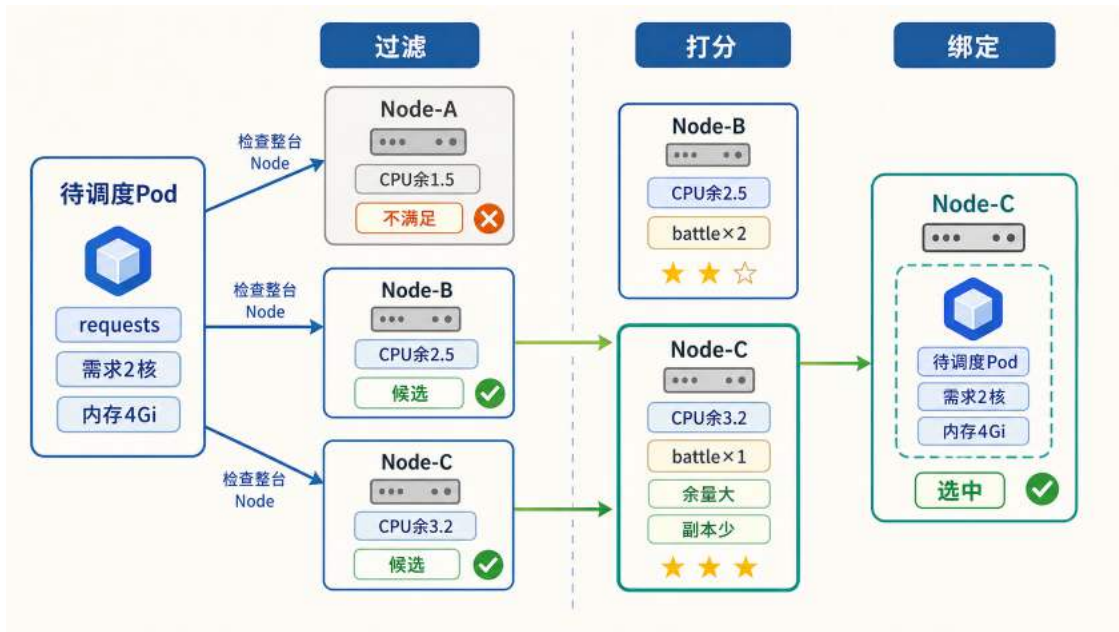


图 67: 调度器的过滤、打分与绑定过程: Node-A 无法容纳完整 Pod 而被排除, Node-B 和 Node-C 进入打分, Node-C 因资源余量更大且已有 `battle` 副本更少而被选中。

这个例子揭示了一个重要约束：调度器比较的不是集群的资源总量，而是每台 Node 能否独立容纳整个 Pod。Node-B 和 Node-C 都通过了资源门槛，但 Node-C 放置后仍有更大余量，且已有同类副本更少，因此更符合分散放置的目标。这些约束并不意味着默认调度器能够找到全局性能最优的放置结果。调度器依据工作负载已经声明的资源需求、拓扑要求和评分规则，从当前可行节点中选择目标；端到端延迟、数据本地性或专用硬件利用率等目标，可能还需要更精确的指标、专用调度策略或自定义调度器。资源声明与拓扑约束提供的是可执行的放置条件，而不是对运行性能的完整预测。

资源碎片 上述约束带来了一种常见的调度失败场景：集群中所有 Node 的剩余资源总和足够承载一个新 Pod，但没有任何一台 Node 单独满足 Pod 的 `requests`。例如 `battle` 的每个副本需要 2 核 CPU 和 4GiB 内存，但集群中三台 Node 分别只剩余 1.5 核、1.2 核和 1.8 核 CPU。三

台 Node 的余量之和为 4.5 核，远超单个副本的需求，却没有任何一台达到 2 核的门槛。此时控制器已经发出补充副本的指令，但 Pod 停留在 Pending 状态无法运行。从集群监控面板看整体利用率并不高，但玩家侧已经感知到匹配队列变长、开局延迟上升。

这种现象称为资源碎片。其根本原因在于 Pod 必须完整地运行在一台 Node 上，不能跨 Node 拆分资源。集群的有效可调度容量不等于各 Node 剩余资源的简单加总。

分散放置与拓扑约束 资源约束决定 Pod 能否被放置，但放置决策还需要考虑故障隔离。如果同一个 Deployment 的多个副本被集中放置在同一台 Node 上，该 Node 宕机时所有副本同时消失。以 `battle` 为例：6 个副本中若有 4 个运行在 Node-B 上，Node-B 故障会导致 4 个副本同时不可用，正在进行的对局大面积中断。如果副本均匀分散到 3 台 Node 上，同样的单机故障只影响 2 个副本，冲击范围更可控。

K8s 提供两类相关机制。Pod 反亲和 (Pod Anti-Affinity) 描述哪些工作负载不应集中在同一故障范围内，可以把同类副本分散到不同节点。拓扑分布约束 (Topology Spread Constraints) 进一步描述副本在节点或可用区之间应当保持多大程度的均匀，例如限制不同可用区的副本数差值不能超过给定范围。前者回答“哪些 Pod 应当彼此分开”，后者回答“副本应当在多个故障范围之间分布得多均匀”。二者共同降低单个节点或单个可用区故障时同时丢失大量副本的风险。

调度作为控制面的前置环节 控制器决定“做什么” (补副本、替换版本)，调度器决定“放在哪台 Node 上”。但调度器本身并不直接操作节点，它只把调度结果写回 API Server；真正在目标节点上创建容器的是 kubelet。那么，API Server、etcd、Controller Manager、Scheduler 和 kubelet 之间是如何协作的？这正是 Kubernetes 控制面要回答的问题。

5.2.6 Kubernetes 的控制面与协作链路

Swarm 的 Manager 将 API 接收、状态存储、调度和健康检查四项职责集中在同一类角色中。K8s 将这四项职责拆分为独立的组件，每个组件只承担一项任务，组件之间通过统一的枢纽协调，而不是在同一进程内直接调用。

K8s 的核心控制面通常由四类组件构成。**API Server** 是集群中所有操作的唯一入口：部署、扩容、查询、删除等一切请求都必须经过它的接收和验证，其他组件不直接相互通信，而是统一通过 API Server 读写集群状态。**etcd** 是一个独立部署的分布式键值存储，作为 API Server 的后端存储，保存集群中的资源对象和状态信息。将存储独立出来意味着即使 API Server 或其他组件重启，集群的配置信息和运行记录不会丢失。**Scheduler** 只负责放置决策：当一个新的工作负载需要运行时，它根据各节点的 CPU 和内存余量、亲和性规则和约束条件，为其选择一个合适的目标节点。选完之后 Scheduler 的工作就结束了，它不参与后续的容器启动过程。**Controller Manager** 负责“期望状态与实际状态持续对齐”这一控制循环。前面在 Swarm 中已经接触过这种思路，K8s 将其进一步细化为多个独立控制器，每个控制器负责一类资源的对齐：Deployment 控制器持续检查副本数是否达标，Node 控制器持续检查各节点是否存活。控制器发现偏差时不直接操作节点，而是向 API Server 提交纠正请求，由后续组件完成执行。

节点执行面的职责由 **kubelet** 承担。每个工作节点上运行一个 kubelet 进程，它持续从 API Server 获取分配给本节点的容器列表，调用容器运行时完成创建或销毁，并向 API Server 汇报节点和容器的健康状态。kubelet 是控制面决策的最终执行者。

Swarm 与 Kubernetes 都把控制职责和节点执行职责分开，但 Kubernetes 进一步拆分了状态入口、持久化、调度和状态调谐。图 68 用两种控制结构说明这种职责粒度的差异，而非比较产品优劣。

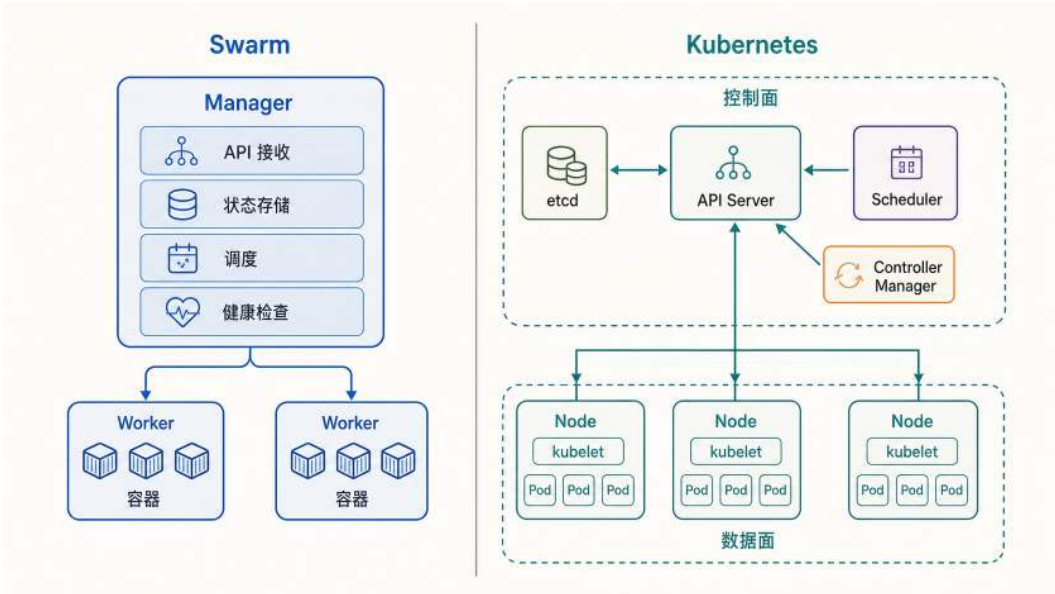


图 68: Swarm 与 Kubernetes 控制结构对比: Swarm 的 Manager 角色集中承担主要控制职责, Kubernetes 将其拆分为 API Server、etcd、Scheduler 和 Controller Manager 等独立组件, 工作节点由 kubelet 执行控制面形成的放置结果。

以上是各组件的独立职责。为了看清它们如何协作，以一个具体场景为例：运维团队决定将战斗服从 2 个副本扩容到 4 个副本。

在 Swarm 中，运维人员执行 `docker service scale battle=4`，请求进入 Manager 后，由 Manager 角色内的编排逻辑统一完成验证、状态更新、节点选择和任务下发。

在 K8s 中，同一个扩容意图经过的路径截然不同。运维人员执行 `kubectl scale deployment battle --replicas=4`，请求首先到达 API Server。API Server 验证请求合法后，将“期望副本数 = 4”写入 etcd。此时集群状态已变更，但还没有任何实际动作发生。

接下来由 Controller Manager 接手。它内部的 Deployment 控制器通过 API Server 观察 Deployment 的状态，检测到期望副本数为 4 而当前副本不足，于是创建新的 Pod 对象来补位。这些 Pod 处于“待调度”状态，尚未绑定到具体节点。

Scheduler 随即检测到存在待调度的 Pod，为每个 Pod 选择目标节点，并将调度结果写回 API Server。至此，控制面的决策环节全部完成，剩下的是执行。

目标节点上的 kubelet 从 API Server 获取到本节点新增的 Pod 信息，拉取战斗服镜像、创建容器并启动，随后持续向 API Server 汇报运行状态。从运维人员发出命令到新副本上线，整个过程由多个组件接力完成，每个组件只处理自己职责范围内的一步。

这个过程中有一个关键的设计原则：每个组件只在自己的职责范围内行动，通过修改 API Server 中的状态来触发下一个组件的响应。Controller Manager 不向 Scheduler 直接发出调度指令，Scheduler 也不向 kubelet 直接发出启动容器的命令。所有协作都通过状态变更驱动，而非直接的命令调用。这种松耦合使得单个组件可以独立重启或升级，短暂的不可用不会导致其他组件连锁故障：重启完成后，组件从 etcd 中读取当前状态即可恢复工作。

对于游戏后端运维而言，这种架构打开了 Swarm 无法提供的可能性。Scheduler 可以替换为自定义实现：例如将同一房间内的战斗服副本调度到网络延迟最低的节点组合上，这在 Swarm 的固定调度逻辑中做不到。etcd 独立部署为高可用集群后，状态存储不再绑在某一个控制角色内部，而是成为可单独维护和备份的基础组件。控制面的可拆分设计还意味着团队可以根据业务需要替换或扩展其中的某个组件，而无需修改平台本身的代码。

至此，对象定义、协作关系和放置决策已经建立。但上述协作只说明了“一次扩容”的执行路径。Deployment 的副本数声明为什么能被持续维持？Pod 故障后为什么能自动补位？下一节将分析声明式管理与调谐循环。

5.2.7 声明式管理与调谐循环

多机环境下，发布与运维最棘手的问题不是“如何执行操作”，而是**操作执行到一半失败后，系统到底处于什么状态**。

考虑一个典型场景：lobby-api 在高峰时段出现间歇性超时，运维人员决定紧急发布热修版本。发布脚本按顺序执行：拉取新镜像、逐台重启进程、更新后端列表。替换进行到第三台时，一台 Node 短暂不可达，两个实例拉镜像失败，另外三个新实例启动后未能通过健康检查。此时玩家侧的表现是：部分用户能正常登录，部分用户持续等待，部分用户进入房间后偶发失败。故障难以归因，因为系统既不是完全可用，也不是完全不可用，而是处于一种无法描述的中间状态。

这类故障的根源在于命令式发布的基本假设：脚本描述的是步骤序列，每一步依赖前一步的成功。一旦中途失败，系统无法回答“哪些步骤已生效”“哪些步骤被跳过”“当前整体是否可用”。为了应对中途失败，运维流程不得不叠加探测、重试、回滚和补偿逻辑。节点越多，失败点越多，这层补偿逻辑的复杂度增长不可控。

解决这一问题的思路是改变描述维度：不再追踪“上一步执行到了哪里”，而是持续回答三个问题。第一，**目标是什么**（例如大厅服期望保持 3 个可用副本）；第二，**现状是什么**（当前有几个副本可用、入口指向了谁）；第三，**差距是什么**（离目标还差几个、哪些需要补建）。一旦以目标与现状的差距作为驱动，系统在任意时刻都只需要缩小当前差距，而不需要追溯历史步骤或判断中断发生在哪一环。

这就是**声明式管理** (Declarative Management) 的出发点：用户提交**期望状态** (Desired State)，平台负责将现实状态持续拉向目标。**调谐循环** (Reconciliation Loop) 是实现这一目标的执行机制：控制器在后台不断重复“观察现状 → 对比差距 → 执行纠偏 → 再次观察”的过程。它将“失败”重新定义为“与目标之间的偏差”，不再依赖人工判断应该补执行哪一步。只要纠偏动作满足幂等性（对同一目标重复执行与执行一次的效果相同），系统就能在多次尝试后收敛到目标状态。

声明式管理成立的关键是目标状态能够被持久化，纠偏动作通过统一状态接口接力，实例是否接收流量则由就绪状态决定。故障发生后，同一组组件重新进入循环，而不需要恢复某条命令执行到一半的位置。图 69 将控制面决策与节点执行连成这一持续纠偏闭环。

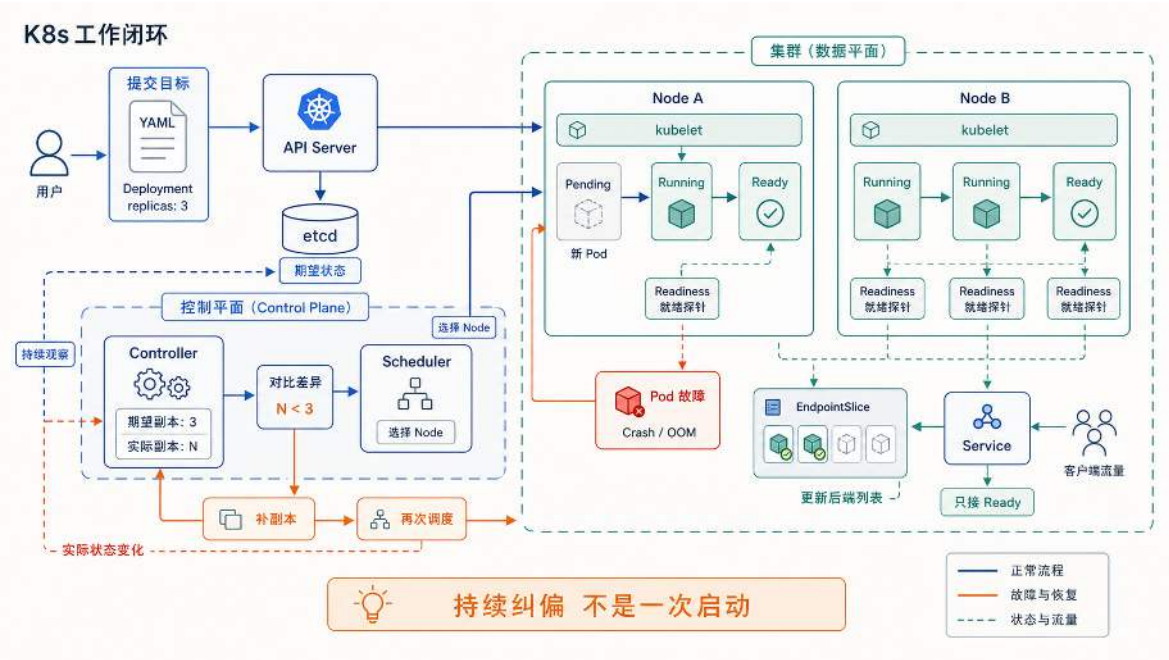


图 69: K8s 工作闭环：用户提交目标后，控制面持续观察、对比差异、调度补位，数据面执行就绪检查并更新后端列表，形成完整的持续纠偏回路。

控制闭环建立后，命令式与声明式管理的差别集中在恢复责任上。图 70 将两种模式置于同一中途失败场景，比较由谁判断现状并继续恢复。

命令式流程一旦中途失败，运维人员必须人工判断应从哪一步恢复；声明式流程则将目标状态持久化在控制面中，由控制器围绕“观察现状”“对比差距”“执行纠偏”形成持续回路，发现现实状态偏离目标时自动纠偏。这种持续纠偏的行为类似恒温器的工作方式：用户设定目标温度，系统根据当前温度决定加热或停止，具体操作几次、每次持续多久由控制回路自行决定。对应到 K8s 中，目标温度就是 Deployment 的期望副本数，当前温度就是实际运行的 Pod 数量，加热或停止就是创建或删除 Pod 的纠偏动作。

在这个模型下，平台的自愈能力有了清晰的因果链：期望状态被持久化在控制面（etcd）中，控制器持续对比现实与期望，偏差触发纠偏动作。Deployment 负责将副本数量和版本拉回目标，Service 负责提供稳定入口并维护后端 Pod 列表。两条因果关系构成自愈的基础：目标被显式记录，以及平台持续将现实拉向目标。

5.2.8 调谐循环的运行行为

本节以 lobby-api（大厅服）为例，分析声明式约束在运行时的具体行为。假设初始状态如下：Deployment 声明 replicas: 3，三个 Pod 分布在不同 Node 上，Service 通过标签选择器定义访问范围，Service 将匹配且就绪的 Pod 纳入其后端列表，调用方通过 Service 名称 lobby-api

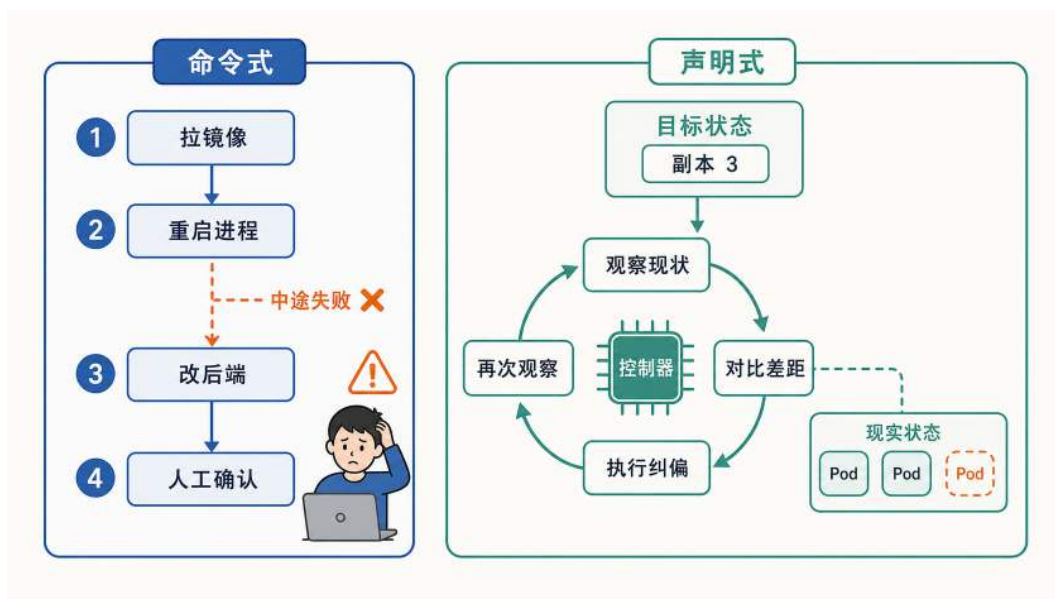


图 70: 声明式管理与命令式运维的对比：前者描述目标并持续纠偏，后者依赖按步骤执行和人工补偿。

访问大厅服。从这一稳态出发，下面依次考察副本故障、版本升级和启动就绪三个场景。

场景一：副本异常退出 假设其中一个 Pod 因内存溢出 (OOM) 被操作系统终止。此时集群中还剩两个正常运行的 Pod，第三个已经消失。Deployment 控制器在下次对比中发现实际副本数 (2) 低于期望值 (3)，随即创建一个新 Pod，调度器为其选择目标 Node 后由 kubelet 启动。新 Pod 启动并通过 Readiness Probe 后，其就绪状态会反映到 Service 的后端列表中；与此同时，已终止的 Pod 会从该列表中移除。对调用方而言，整个过程中访问的入口始终是 Service 名称 lobby-api，后端成员的变化不影响调用方的配置和代码。

这个过程揭示了副本数约束的关键性质：它不是部署时执行一次的指令，而是控制器持续维持的不变式。只要实际值偏离期望值，补位动作就会被触发，无论偏离的原因是节点宕机、进程崩溃还是人工误删。如果 Pod 并未完全终止，而是处于进程假死状态（进程仍在但不再响应请求），livenessProbe 的连续失败会触发 kubelet 重启容器，随后 Deployment 控制器检查副本数是否恢复达标。无论触发原因是 OOM 终止还是 livenessProbe 重启，调谐循环的最终效果都是将实际状态拉回期望状态。

场景二：滚动升级 假设大厅服需要从 v1 升级到 v2。运维人员将 Deployment 中的镜像字段从 game-lobby:v1 改为 game-lobby:v2，控制器随即开始执行升级。如果一次性终止全部 3 个旧副本再启动新副本，升级期间没有可用实例，玩家侧表现为集中断线和登录失败。滚动更新策略避免了这种中断：控制器将升级拆分为多轮，每轮只替换一部分副本，在任意时刻都保持足够数量的可用实例承载流量。替换节奏由两个参数控制：maxSurge 规定过程中允许临时超出目标数量的上限，maxUnavailable 规定允许暂时不可用的上限。在前面 YAML 示例的配置下 (maxSurge=1, maxUnavailable=0)，控制器先启动一个新版本 Pod，待其就绪后再终止一个旧

版本 Pod，如此循环直到 3 个副本全部替换完毕。maxUnavailable=0 保证在整个过程中始终有不少于 3 个就绪副本承载流量。

滚动更新将升级从瞬时切换变为可控过程：Deployment 管理的不只是最终的目标版本，还包括到达目标版本的过程中可用性的下限。

场景三：启动但未就绪 场景一和场景二都涉及新 Pod 的创建。一个容易被忽略的问题是：Pod 的进程启动成功，不代表它已经能够正常处理请求。大厅服启动后通常还需要加载配置、建立数据库连接池、预热本地缓存，这些初始化步骤可能耗时数秒。如果平台在进程刚启动时就把该 Pod 标为 Service 的可用端点，调用方的请求会被转发到一个尚未准备好的实例上，导致超时或错误。

Readiness Probe 解决的正是这个问题。它在 Pod 与 Service 之间充当闸门：kubelet 按配置的频率（前面 YAML 中为每 5 秒）向 Pod 发送探测请求，只有探测成功后，该 Pod 才会在 EndpointSlice 中成为可接收流量的端点。未通过就绪检查的 Pod 不会接收 Service 流量，启动阶段的不确定性不会传导到调用方。

Readiness Probe 与 Liveness Probe 是两个独立的判断：Liveness Probe 判断进程是否仍能正常运行、是否需要被重启；Readiness Probe 判断进程是否可以接收流量、加入服务。一个 Pod 可能处于“存活但未就绪”的状态（正在初始化），也可能已经通过就绪检查，但随后因死锁、内存泄漏或探测失败被 Liveness Probe 重启。两个探针各自独立工作，分别触发不同的平台动作。

三个场景共同展示了声明式约束在运行时的行为模式：副本数约束驱动控制器持续补位，滚动更新约束控制替换节奏，就绪探针将未准备好的实例隔离在入口之外。系统是否已收敛到期望状态，可以通过两个可观测指标判断：副本数是否达标，以及 Service 后端列表中可用端点的数量和状态是否符合预期。

将同样的分析框架应用到 **battle**（对局服务）时，约束之间的关系会更复杂。副本不足时匹配队列增长、开局延迟上升；滚动替换时如果没有排水机制，正在进行的对局可能被中断。平台能维持副本数量和控制替换节奏，但对局是否允许被中断、中断后状态如何恢复，必须由业务逻辑定义。平台提供基础设施层面的保障，业务语义层面的约束需要应用代码配合实现。

上述三个场景都以已经声明的副本数和版本为目标，调谐循环负责在状态偏离后恢复这一目标。负载持续变化时，系统还需要动态判断目标副本数和底层节点容量应当如何调整，管理对象由“维持既定状态”进一步扩展为“根据运行信号修正目标状态”。

5.3 弹性运行：伸缩与自愈

编排已经解决了多服务在集群中的启动、互联、副本维护和放置问题。但线上系统还要面对两类持续变化：负载升降会改变所需容量，进程或节点故障会破坏现有实例。平台需要在这些变化中维持可度量的服务质量，同时控制资源成本。平台可以为延迟、错误率、可用性或队列等待时间等指标设定目标值，这类目标称为服务级别目标（Service Level Objective, SLO），并为容量调整和故障恢复提供判断标准。弹性运行由两条相互关联的链路组成：自动伸缩根据负载调整

实例与节点数量，故障自愈根据健康状态修复失效实例。前者处理容量偏差，后者处理健康状态偏差，共同服务于既定的 SLO。

5.3.1 伸缩调度：固定容量与波动负载的矛盾

在线系统的负载通常具有时间结构：活动开场、促销开始、协作高峰或版本更新上线都可能在短时间内使请求量成倍上升，而低谷时段又可能长时间处于低利用率。固定容量以空闲成本换取即时承载能力，弹性容量减少低谷浪费，却引入了检测和扩容时延。图 71 将这一取舍置于同一负载时间轴上；当负载上升速度超过扩容速度时，请求积压和尾部延迟仍可能被放大。

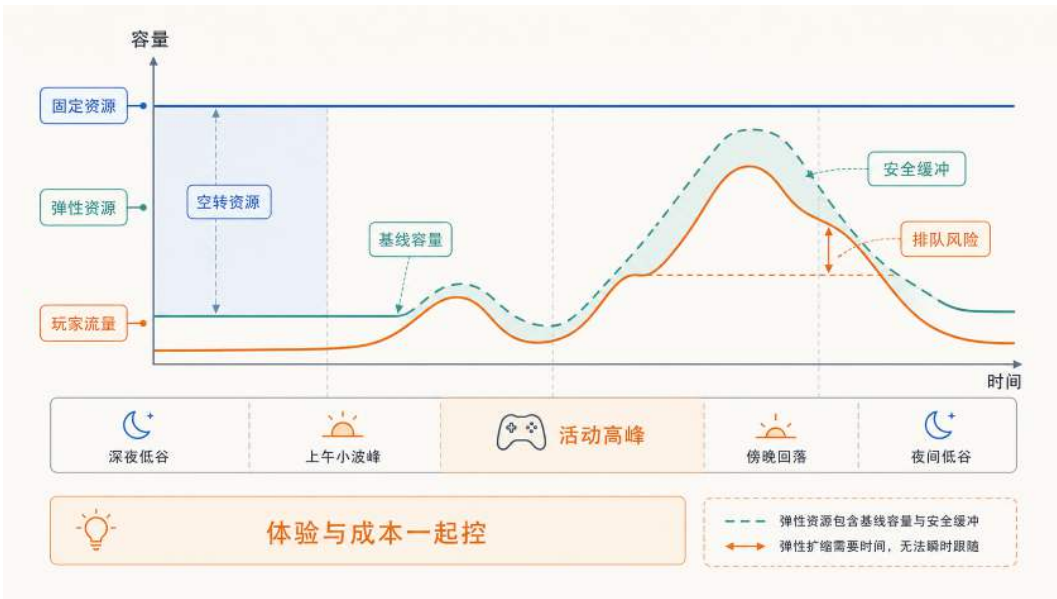


图 71：固定容量与弹性容量的资源利用对比：固定容量按峰值预留时会在低谷期产生空转浪费；弹性容量根据实际负载动态调整，并通过安全缓冲吸收高峰波动，但扩缩容存在时间滞后，突发变化仍可能带来排队风险。

以一个具体场景说明这种矛盾的量级。假设活动高峰期同时在线 5 万人，需要 50 台服务器支撑；而工作日上午在线人数降至 5000，5 台即可满足。如果始终维持 50 台，90% 的时间中 90% 的资源处于空转状态。如果按平均负载配置（例如 15 台），高峰期的请求到达速率将远超处理能力，客户端出现连接超时和响应失败。

这一矛盾不仅来自周期性的峰谷波动，还来自不可预测的突发事件。一次外部推荐或运营活动可能在数分钟内使请求量增长数十倍，远超任何基于历史均值的静态预估。突发流量对玩家体验的冲击比持续性高负载更严重：大量用户在同一时间窗口内集中遭遇登录失败或操作超时，对服务可用性的感知迅速恶化。

非线性放大：排队论视角 容量不足的后果并非线性增长。当请求到达速率逼近服务处理能力的上限时，队列长度和尾部延迟可能迅速放大。其原因是每个请求的处理时间存在波动：当系统缺少空闲处理能力时，偶发的慢请求无法被及时消化，积压逐步累积，后续请求的等待时间随队列增长而延长。

排队论中的一个经典结论是：当系统利用率接近 1 时（到达速率接近处理速率），平均等待时间会随利用率急剧上升。利用率越高，可用于吸收服务时间波动的空闲容量越少，偶发的处理延迟越容易转化为持续积压。

因此，扩缩容不能只在错误率升高后才启动，还应关注排队长度和等待时间等更早出现的过载信号。第 6.4.2 节将结合 Little 定律说明这些信号如何进入弹性控制闭环。

体验恶化的时间顺序 将上述机制投射到一次活动开场的时间线上，可以观察到体验恶化的典型顺序：首先是请求到达速率和活跃连接数上升；当到达速率超过处理能力后，请求开始排队；排队累积导致响应时间逐步拉长，P95/P99 等尾部延迟指标率先恶化；当积压持续加深，部分请求的等待时间超过客户端设定的超时阈值，错误率开始上升。

这一顺序意味着：当客户端开始出现响应缓慢时，系统往往已经进入排队累积阶段，但尚未触发错误率告警。弹性伸缩的目标是在排队累积的早期阶段介入，在尾部延迟恶化传导到错误率之前补充处理能力。

但负载的变化是持续且不可预测的：高峰可能在几分钟内到来，也可能在活动结束后迅速回落。一次性的手工扩容只能应对已经发生的过载，无法跟随后续的波动。要在变化的负载下持续维持容量与需求的匹配，系统需要一种能够反复执行“观察当前状态 → 判断是否偏离目标 → 调整容量 → 再次观察”的机制。这种持续修正偏差的结构在控制论中称为反馈闭环（feedback loop）。Kubernetes 的弹性伸缩正是基于这一机制构建的。

5.3.2 应用层自动扩缩：HPA 与 VPA

在手工运维模式下，扩容是一组顺序操作：启动新的服务器实例，部署同版本的服务，将新实例注册到负载均衡的后端列表；缩容则需先将目标实例从后端列表中移除，等待其上的存量请求或会话结束，最后终止进程。整个过程由运维人员判断时机、执行动作、观察效果，本质上是一个人工驱动的控制过程。

Kubernetes 通过 Horizontal Pod Autoscaler (HPA) 将这一过程自动化。运维人员不再逐步操作，而是声明一个目标状态（例如“将 lobby-api 的平均 CPU 利用率维持在 60% 附近”），HPA 持续将实际状态向目标靠拢。从使用者角度看，HPA 需要声明目标指标、目标值以及最小和最大副本数；平台随后反复完成指标采集、容量计算、实例创建或回收和结果再观察。控制器怎样处理容忍区间、稳定窗口和执行滞后，将在第 6.4 节从内部机制展开。

计算逻辑 HPA 的核心计算逻辑是按比例换算：当前指标高于目标时增加副本，低于目标时减少副本，偏差越大，期望调整幅度通常也越大。具体计算并不等同于容量立即生效，新副本仍需经过调度、启动和就绪检查。

信号选择 HPA 的决策质量取决于输入信号与玩家体验之间的相关性。CPU 利用率是最常用的默认指标，优点是获取成本低且无需应用改造，但它与用户可感知的延迟之间并非线性对应。对入口服务，更直接的信号可能是活跃连接数、请求到达速率（QPS）或 P95/P99 延迟；对匹

配服务，等待队列深度比 CPU 更能反映用户等待时间；对战斗服务，活跃房间数和逻辑帧耗时与玩家体验的关联更紧密。

信号选择的本质是确定闭环保护的对象：选择 CPU 意味着闭环保护的是计算资源余量，选择请求延迟意味着闭环直接保护用户响应时间。当 CPU 尚有余量但延迟已经升高时（瓶颈在网络 I/O 或下游依赖），以 CPU 为信号的 HPA 不会触发扩容，用户体验持续恶化。因此，信号的选择决定了闭环能覆盖哪些过载场景。

闭环的两类约束 HPA 的工程实现面临两类固有约束。第一类是**信号噪声**：负载指标存在短时波动，如果控制器对每次波动都响应，系统会频繁扩缩产生振荡。第二类是**执行时延**：从计算期望副本数到新 Pod 实际承接流量，中间还要经历调度、镜像拉取、进程启动和就绪检查，耗时可能从数秒到数十秒。两类约束共同决定了灵敏度设置的核心矛盾：过高则振荡，过低则滞后。控制器如何通过容忍区间、稳定窗口和冷却时间在这之间取得平衡，属于平台内部的实现细节，将在第 6.4 节展开。

扩容的边界 HPA 通常配置最小和最大副本数：最小值确保低负载时段仍保留基本冗余，最大值受集群资源和成本预算约束。即使 Pod 数量增加到最大值，如果性能瓶颈位于下游依赖（数据库连接池耗尽、缓存命中率下降、某个分区成为热点），增加入口层副本无法缓解这些瓶颈。此时需要配合限流、降级或故障隔离等手段保护下游依赖，而非继续扩容入口层。

另一条路径：垂直扩缩 (VPA) HPA 沿水平方向调节容量：负载升高就增加副本，负载回落就减少副本。但有一类容量问题不是副本数不够，而是单个副本的资源规格本身不合适。如果一个 Pod 的内存 `requests` 估得过低，它会在负载升高时频繁触发 OOM 被重启；估得过高，则每个副本长期占用超过实际需要的资源，压低整台 Node 的部署密度。这类问题靠增减副本数无法解决，需要调整的是单副本的 `requests` 和 `limits`。承担这一职责的机制是垂直扩缩 (Vertical Pod Autoscaler, VPA)：它观察 Pod 的历史资源使用情况，自动调整其资源请求与上限，使规格贴近真实消耗。VPA 与 HPA 的分工可以这样区分：HPA 回答“需要多少个副本”，适合可水平扩展的无状态服务；VPA 回答“每个副本该配多大”，适合副本数不宜频繁变动、但规格需要校准的服务（如前面提到的 Redis、PostgreSQL 这类有状态组件）。需要注意的是，VPA 自动改变规格通常要重建 Pod（新规格只能在新 Pod 上生效），对在线服务意味着一次重启。因此在延迟敏感的实时链路上，VPA 更常配置为只给出资源建议、由人工在发布窗口采纳，或仅用于可以承受重启的非实时服务；真正“边运行边改规格”的原地调整能力仍受运行时和平台支持程度限制。

均值掩盖局部过载 HPA 的默认决策基于所有副本的指标均值。当负载分布不均时（例如少数副本因会话粘性或热点用户承受了远高于均值的负载），均值可能仍在目标范围内，HPA 不会触发扩容。结果是多数用户体验正常，少数用户持续遭遇高延迟。要覆盖这类局部过载场景，需要使用更精细的指标（如分位数指标或单副本最大值）或结合负载均衡策略调整流量分配。

HPA 在扩容方向的逻辑相对直观：负载高就加副本。缩容方向的问题更为复杂：扩容不足的后果是响应变慢，而缩容失误的后果可能是用户连接直接中断。

5.3.3 安全缩容：为什么撤掉容量更危险

扩容不足的后果主要是响应变慢：玩家等待时间增加，已建立的连接通常不会立即中断。缩容失误的后果则严重得多：如果一个正在处理请求或承载会话的 Pod 被直接终止，用户看到的是连接断开、对局中断或数据丢失。因此，缩容的工程复杂度不在于计算“应该减少多少副本”，而在于如何安全地将一个正在服务的副本从系统中移除。

安全缩容不是直接删除实例，而是先改变流量状态，再排空存量工作，最后回收容量。图 72 建立了这一不可颠倒的顺序。

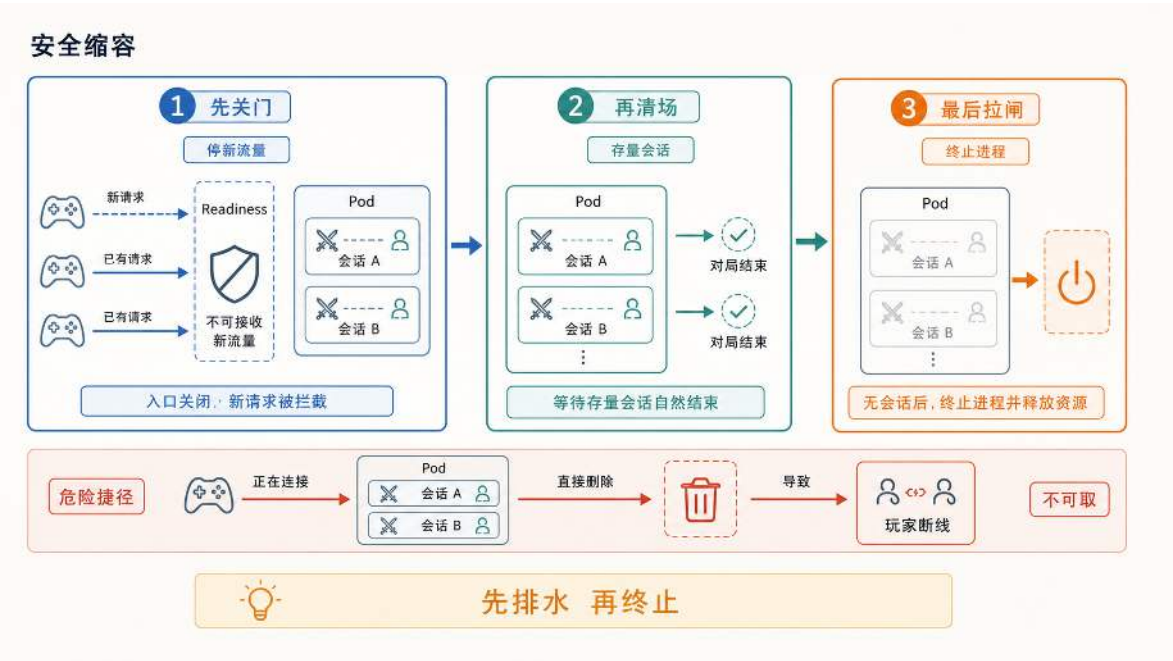


图 72: 安全缩容的基本顺序：先停止接收新流量，再等待存量请求或会话结束，最后终止进程。

三个阶段依次是：停止接收新流量，将实例从服务入口中移除；等待存量请求或会话自然完成；最后终止进程并回收资源。对短请求服务，排空可能只需毫秒级；对有状态或长连接服务（如承载对局的战斗服），则需要等待对局结束或配合会话迁移，耗时远更长。平台提供优雅终止的时序保证，但“实例上的工作何时可以安全中断”属于业务语义，需要应用层定义退出条件。平台机制与应用退出逻辑如何配合，将在第 6.3 节从应用设计角度展开。

HPA 在负载回落时会计算出更小的期望副本数，触发上述缩容流程。但 HPA 能够扩容的前提是集群中存在足够的可用资源。当所有 Node 的剩余资源都无法满足新 Pod 的 requests 时，Pod 会进入 Pending 状态，扩容在 Pod 层面无法继续推进。解决这一问题需要将扩缩从 Pod 层面延伸到 Node 层面。

5.3.4 基础设施层自动扩缩：Cluster Autoscaler

HPA 决定服务需要多少 Pod 副本，但副本能否实际运行取决于集群是否存在满足资源、标签和拓扑约束的 Node。Pod 无法调度时，Cluster Autoscaler 会评估预先配置的节点组；只有新增某类 Node 确实能够使 Pod 可调度，才会向云平台申请机器。Node 创建和加入集群通常比 Pod 启动更慢，缩容还要确认工作负载能够迁走，因此工程上仍需保留一定节点余量。HPA、调度器与节点扩容器怎样分层协作，将在第 6.4 节展开。

容量规划：基线与按需的两段式策略 弹性机制解决了容量如何跟随负载变化的问题，但同时引入了一个成本决策：常驻多少容量、按需扩展多少容量。

按峰值一次性准备服务器时，只要日常负载显著低于峰值，就会产生长期闲置。云平台提供按使用量付费的计费模型，可以减少为波动负载长期保留资源的成本。但前文已经说明，Pod 和 Node 的扩容都存在启动与就绪时延。如果将常驻容量压到极低，每次负载上升都必须等待扩容完成，用户在等待期间会经历排队和延迟升高。

因此，工程上通常采用两段式容量规划：用一部分常驻容量覆盖可预期的日常负载（基线），用弹性扩缩覆盖超出基线的波动部分。基线的作用是确保日常负载波动不触发扩容延迟，使 HPA 只需要应对超出日常范围的突增。以 `lobby-api` 为例，如果日常峰值稳定在 8 个 Pod，基线可以设为 6 到 8 个副本（HPA 的 `minReplicas`），活动期间的额外负载由 HPA 扩容至 20 到 30 个副本承接。

基线设置过低，系统在每次日常波动时都会触发扩容和缩容循环，带来不必要的延迟抖动和资源振荡。基线设置过高，低谷时段大量副本处于空闲状态，资源利用率下降。从反馈闭环的角度看，基线相当于将系统的稳态工作点设在目标水位附近，减少控制器需要追赶的偏差幅度。

云资源采购模型 两段式策略的成本优化依赖于常驻容量与按需容量的配比。常驻容量适合覆盖可预期的日常负载，单位成本较低但灵活性差；按需容量适合应对波动，单位成本较高但可按需释放。具体计费模型因云平台而异，核心取舍是：常驻比例越高，扩容延迟越小，但低谷空转成本越大；常驻比例越低，成本越灵活，但高峰扩容等待时间越长。

对游戏场景，典型的配比思路是：将长期在线的基线并发（入口层、匹配服务、数据库代理）放在成本较低的常驻实例上，将活动期间的临时扩容放在灵活的按需实例上，将非实时的运维任务（日志归档、离线统计）放在价格更低但可能被回收的竞价实例上。

5.3.5 函数粒度的伸缩调度：Serverless

Serverless（无服务器计算）是一种与常驻服务不同的执行模型：开发者将业务逻辑以函数或事件处理器的形式交付给云平台，平台负责准备执行环境、运行代码、返回结果，并在空闲时回收资源。”无服务器”并不是没有服务器，而是开发者不再直接管理服务器实例、操作系统和扩缩容流程。计费通常按代码实际执行的时间、调用次数以及配置的预热容量计算，而非按长期租用服务器计费。在没有事件触发且未配置预置并发或最小实例数的普通按需模式下，代码不运行，费用接近零。

与前文讨论的 Kubernetes 弹性伸缩相比，Serverless 在控制粒度上有本质区别。HPA 的控制对象是常驻 Pod 的副本数：观察到负载上升后增加副本，观察到负载下降后减少副本，但最小副本数通常不为零。Serverless 的控制对象是事件触发的执行实例：请求或事件到达时，平台优先复用已经准备好的热实例；当没有可用实例或并发容量不足时，再创建新的执行环境。处理完成后，实例可能被保留一段时间等待复用，也可能在空闲窗口后被回收。在允许 scale-to-zero 且未配置最小实例数的模式下，长时间无请求时实例数可以降为零。这种模型将触发信号从“资源水位偏差”前移到“事件到达”本身，减少了对周期性指标采样的依赖；但实例创建、并发扩容和平台配额仍会带来延迟。

代价是平台必须在极短时间内完成执行环境的创建和初始化。当请求到达时如果没有可用的预热实例，平台需要从零启动一个新的执行环境（冷启动），这一过程引入了额外延迟。冷启动的时延和 Serverless 的适用边界是本节讨论的核心问题。

从常驻服务到按需函数 部署模型的选择本质上是在常驻成本与首次请求延迟之间取舍。图 73 将 Kubernetes 的常驻服务模式与 Serverless 的按需执行模式置于同一资源占用条件下比较。

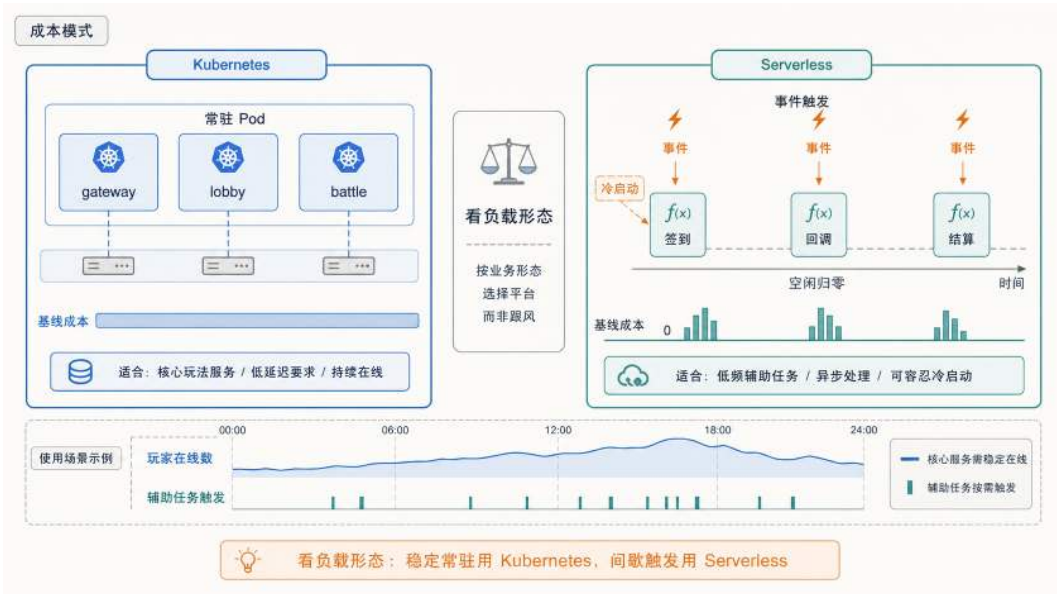


图 73: Kubernetes 与 Serverless 的成本模式对比：核心服务常驻以保证低延迟，低频任务可按事件触发；在允许归零且未配置预热容量时，空闲期可以释放运行实例。

在 Kubernetes 常驻模式下，服务以 Pod 形式持续运行，即使没有请求也占用 CPU 和内存配额；运维团队需要持续关注集群健康、Node 规格、镜像更新和安全补丁。对核心链路（入口层、匹配服务、战斗服），这种常驻投入是保证低延迟的必要成本。但对低频辅助功能（签到奖励发放、支付回调、排行榜定时结算），绝大多数时间 Pod 处于空闲状态，运维开销与实际计算量严重不匹配。

在 Serverless 按需模式下，开发者将业务逻辑封装为函数并上传到平台。平台在事件到达时调用函数，优先使用可复用的热实例；当并发请求超过热实例容量时，平台再创建新的执行实例。

处理完成后实例可能被保留一段时间以复用，空闲超过平台设定的回收窗口后被释放。若平台允许归零且没有配置预置并发或最小实例数，没有事件触发时通常不存在运行中的业务实例；若为了降低冷启动配置了预热容量，则空闲期仍会保留实例并产生相应成本。

以每日签到为例。玩家的签到行为集中在早晨和晚间高峰，其余时间调用量接近零。如果实现为常驻服务，需要配置 HPA 最小副本数、维护 Deployment 和监控，低谷时段副本空转。如果实现为 Serverless 函数，平台在玩家点击签到时复用热实例或创建新实例，计算并发放奖励，返回结果后实例进入空闲等待复用。高峰期平台自动创建更多并发实例，高峰过后实例逐步回收。运维团队不再直接维护这组空闲副本，但仍需要关注并发配额、下游容量、错误重试和成本上限。

下面用伪代码展示这个签到函数的完整结构。与常驻服务的关键区别在于：整个文件没有启动 HTTP 服务器的代码，没有路由注册，没有监听端口。开发者只需要导出一个处理函数，平台负责在事件到达时调用它。

签到奖励的 Serverless 函数实现

```
1 package checkin
2
3 // Handler 是平台在每次事件到达时调用的入口函数
4 func Handler(event Event, ctx Context) Response {
5     playerID := event.Body["player_id"]
6     today := ctx.CurrentDate // "2026-06-27"
7
8     // 幂等写入：同一玩家同一天只能签到一次
9     key := fmt.Sprintf("checkin:%s:%s", playerID, today)
10    db := GetConnection()
11
12    // InsertCheckin 使用唯一约束或条件写入，成功时才允许发奖
13    tx := db.BeginTx()
14    reward := CalculateReward(playerID, today)
15    ok := tx.InsertCheckin(key, playerID, today, reward)
16    if !ok {
17        tx.Rollback()
18        return Response{Code: 200, Msg: "already checked in"}
19    }
20
21    tx.AddToInventory(playerID, reward)
22    tx.Commit()
23
24    return Response{Code: 200, Reward: reward}
25 }
```

函数接收两个参数：`event` 包含触发事件的具体内容（HTTP 请求体、消息队列消息等），

ctx 包含平台注入的运行时信息（请求 ID、剩余执行时间、当前日期等）。函数返回一个结构化响应，平台将其转发给调用方。示例中，玩家 ID 和日期组成业务唯一键，InsertCheckin 依赖数据库唯一约束或条件写入完成原子占位；只有占位成功的请求才继续发奖并提交事务。这里不能写成简单的”先 Exists 再 Set”：两个重复请求可能同时通过查询，再分别写入奖励，反而破坏幂等性。

与函数代码配套的是一份部署配置，声明函数如何被触发、分配多少资源、超时多久：

签到函数的部署配置

```
1 function:
2   name: daily-checkin
3   runtime: go1.21
4   handler: checkin.Handler      # 包名.函数名
5   memory: 128MB
6   timeout: 5s                  # 单次执行最长时间
7
8 trigger:
9   type: http
10  method: POST
11  path: /api/checkin
12
13 environment:
14  DB_HOST: ${postgres_endpoint}
15  DB_NAME: game_rewards
```

这份配置体现了平台与开发者之间的职责边界。开发者声明运行时版本、入口函数、内存上限和超时阈值；平台根据这些声明决定使用何种沙箱、分配多少资源、何时回收实例。触发器部分将一个 HTTP POST 请求绑定到签到函数：玩家客户端发送 POST /api/checkin 时，平台自动路由到 daily-checkin 函数的一个执行实例。如果将 trigger.type 改为 schedule（如 cron: "0 4 * * *"），同一个函数就变成了定时任务，每天凌晨 4 点由平台主动触发执行。

对比前面用 Kubernetes 部署常驻服务的方式，两者的差异集中在生命周期管理上。Kubernetes 的 Deployment 声明”始终保持 N 个副本运行”，平台持续维护这些副本的存活状态；Serverless 的部署配置声明”事件到达时如何调用函数、资源和超时如何约束”，平台负责执行实例的创建、复用、回收和并发调度。前者需要运维团队关注副本健康、资源利用率和滚动更新；后者把实例生命周期交给平台，但开发者仍需管理触发器、权限、依赖版本、监控告警、幂等逻辑和下游容量。

应用约束 Serverless 平台可能随时回收执行实例，也可能在故障后重新投递事件。因此，函数不能把业务正确性建立在本地内存或本地文件上，带有外部副作用的操作还必须能够识别重复请求。状态外置和幂等性决定了一个功能能否安全地按需创建与重试；具体设计条件将在第 6.3 节

统一说明。

状态外置和幂等使平台能够按需创建和销毁实例，但“按需创建”本身带有延迟代价。当一个函数长时间无调用后，平台会回收所有空闲实例以节省资源；下一次请求到达时，平台必须重新分配执行环境，这个从无到有的准备过程称为冷启动（cold start）。这里先关注它对选型判断的影响；沙箱精简、快照恢复和运行时优化等内部机制将在第 6.4.8 节展开。

签到是典型的适合场景。签到请求执行时间短（查询记录、计算奖励、写入数据库，整条链路通常很短），所有状态存在外部 Redis 和 PostgreSQL 中，调用量集中在早晚高峰且其余时段接近零。早晨第一批玩家可能触发冷启动，但只要首次响应仍在业务可接受范围内，这一额外延迟通常不会破坏签到流程。高峰过后实例逐步回收；在未配置预热容量的按需模式下，空闲期不必维持业务实例常驻。

支付回调也适合。第三方支付平台在玩家完成付款后，向游戏服务器发送一个 HTTP 回调通知。这类请求完全由外部事件驱动、到达时间不可预测、单次执行只需验证签名并写入一条订单记录。回调对响应延迟的要求宽松（支付平台通常允许数秒内返回确认），且自带重试机制（未收到确认会重新投递）。用 Serverless 处理回调，不需要维护常驻服务来等待这些偶发事件，同时重试机制与函数的幂等设计天然配合。

实时对战则不适合。战斗服需要维持与每位玩家的 WebSocket 长连接，持续接收操作指令并广播帧同步数据。一场对局会持续较长时间，期间连接不能中断、状态不能丢失。典型函数平台通常带有单次执行时长、请求网关超时和实例回收等限制，执行环境也不适合承载长期驻留的连接状态。即使某些容器型 Serverless 平台支持较长连接，冷启动、扩容抖动、实例回收和连接迁移仍然会给强实时链路带来额外风险。帧同步对延迟极其敏感，任何无法稳定控制的首次启动时延都可能表现为明显卡顿。战斗服更适合以常驻 Pod 形式运行，由 Kubernetes 保证副本持续存活，并在应用层处理排水和会话恢复。

前面章节讨论的匹配服务同样不适合直接改成按请求启动的函数。这里的匹配服务在进程内存中维护一个连续变化的匹配队列，持续扫描等待中的玩家、计算评分差异并组合对局。如果每次调用都从外部存储加载完整队列、执行匹配、再写回，不仅延迟增加，多个实例还可能并发修改同一份队列。业务可以把队列状态外置并把匹配拆成可协调的短步骤，也可以按分区维护多个常驻队列；但无论采用哪种方式，都必须先解决队列归属和并发更新问题，不能只靠 HPA 增加副本。

从这四个场景可以归纳出判断标准：函数执行时间短、无需维持连接、状态可外置、对首次延迟不敏感的功能适合 Serverless；需要长连接驻留、依赖进程内状态、或对延迟有严格要求的功能应保持常驻。表 8 将这些判断按业务特征归类，供快速查询参考。

表中“突发流量”一行需要补充一个前提：Serverless 的弹性能否兑现，取决于下游是否承接得住并发。平台可以在短时间内拉起大量函数实例，但如果下游数据库的连接池、缓存的吞吐上限或第三方接口的速率限制先被打满，玩家看到的仍然是超时。云厂商通常还会设置单函数并发配额和区域容量边界，这些约束需要在线前通过压测确认，不能仅凭“自动扩缩”的描述做容量假设。

实际项目通常按组件的连接时长、状态位置、调用频率和延迟目标选择部署模型，而不是为

表 8: Serverless 与 Kubernetes 的场景化决策参考

业务特征	推荐方案	主要原因
长连接实时交互	Kubernetes	WebSocket/TCP 长连接需要稳定连接驻留，容器常驻模型更匹配
有状态会话管理	Kubernetes	会话粘性、进程内状态与低抖动路由更容易在 K8s 中实现
低频触发式任务	Serverless	普通按需模式按调用计费，不必长期为空闲实例付费
不可预测突发流量	视任务而定	无状态、可并行、可重试的短任务适合 Serverless；长连接或强状态链路仍应常驻
定时批处理	Serverless	与事件调度天然契合，任务短平快且资源利用率高
延迟极敏感	Kubernetes	冷启动尾延迟风险较高，常驻实例更容易保障严格时延目标

整个系统采用同一种方案。长期有效的划分原则是：延迟敏感或需要维持连接和进程内状态的核心链路保持常驻，短时、无状态、事件驱动的周边任务按需执行。图 74 用游戏后端验证这一划分方法。

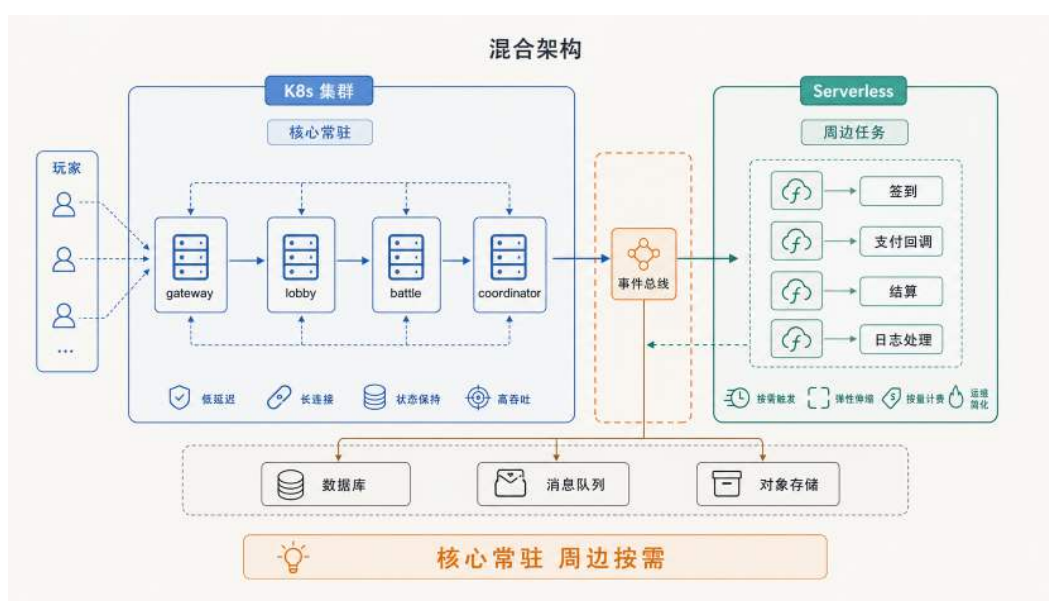


图 74: K8s + Serverless 混合架构：实时连接和对局常驻在 K8s，签到、回调、结算等周边功能按需执行。

常驻部分由 K8s 集群承载：gateway 负责接入玩家长连接，lobby 处理大厅逻辑，battle 运行对局，coordinator 裁决跨服事件。玩家客户端通过长连接接入 gateway，再由 gateway 将请求路由到内部核心服务；延迟和连接稳定性由常驻 Pod 保障。按需部分由 Serverless 平台承载：签到、支付回调、排行榜结算和日志处理均以事件驱动方式执行。两侧通过中间的事件总线连接：当核心服务产生需要异步处理的事件（如对局结束后触发结算、玩家操作触发日志归档），事件总线将其投递到对应的 Serverless 函数。底部的数据库、消息队列和对象存储是两侧共享的基础

设施：核心服务和 Serverless 函数都通过这些外部存储读写状态，函数本身不持有任何本地状态，这正是前面强调无状态约束的落点。

这种混合架构已经回答了” 什么功能放哪里” 的问题，但还留下一个工程细节：Serverless 函数在长时间无调用后，原有执行实例可能被平台回收；如果下一次请求到达时没有可复用的热实例，平台就需要重新创建执行环境。这个重建过程的延迟特征、影响因素和优化思路，是 Serverless 落地时必须量化评估的成本。

冷启动：用首次延迟换常驻成本 冷启动延迟由执行环境、语言运行时和应用初始化等阶段共同累积；暖启动通过复用已有实例跳过其中一部分准备过程。图 75 将两条调用路径置于同一时间顺序中，说明首次延迟与常驻资源成本之间的交换关系。

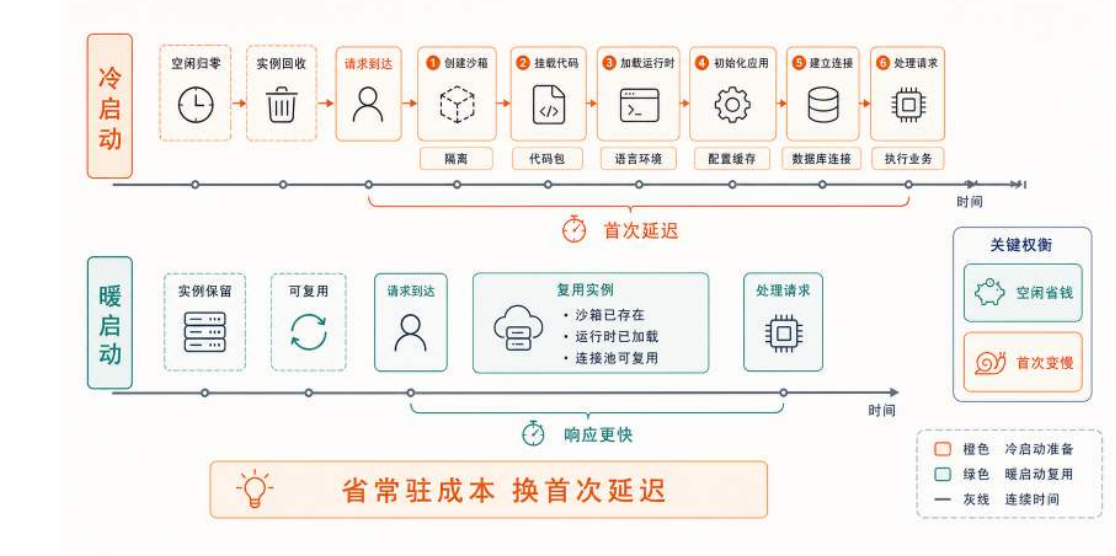


图 75: Serverless 冷启动与暖启动的调用路径对比：冷启动需要重新准备执行环境，暖启动则复用已有实例。

Serverless 最需要评估的问题是冷启动延迟。当平台已有可复用的实例时，请求直接交给现成的执行环境处理，延迟较低，这称为**暖启动**。当函数长期空闲后，平台可能回收原有实例以节省资源，下一次请求到达时如果没有可复用实例，平台就需要重新分配隔离沙箱并启动运行时，这称为**冷启动**。冷启动不是偶然异常，而是 Serverless 用**首次延迟**交换**常驻成本**的自然结果。落到玩家视角，冷启动通常表现为” 第一次特别慢”：例如早晨第一批玩家打开游戏点签到，按钮转圈明显比平时久；又或者某个低频活动入口长时间无人访问，第一次进入时页面加载突兀地卡一下。

除了首次延迟，函数平台通常还会限制执行时间，并可能在故障后重试调用；本地磁盘也不能作为长期状态依赖。这些条件要求业务逻辑能够拆成可独立重试的步骤，并把长期状态写入外部存储。冷启动由哪些阶段组成、MicroVM、快照和 Wasm 分别压缩哪一段成本，将在第 6.4.8 节从引擎视角展开。

5.3.6 故障自愈：检测、修复与约束

前面几节围绕容量变化展开：弹性伸缩让服务在负载起伏时增减副本，Serverless 让低频功能在无请求时归零。这些机制都以实例能够正常工作为前提，调整对象是实例数量。线上环境还会发生进程死锁、内存溢出、节点宕机和网络分区等故障。此时改变的是实例健康状态，单纯增减副本数不能直接修复已经失效的实例。扩缩容解决的是容量随负载变化，故障恢复解决的是实例在运行中失效；两者都用于维持服务目标，但触发信号和处理动作不同。平台自动完成故障检测、修复和恢复验证的过程，通常称为故障自愈（self-healing）。

自动恢复只有在修复动作能够被验证并再次触发检测时，才构成可以持续运行的控制过程。平台需要把故障信号转换为故障层级，在可用性、依赖状态和排水条件允许的范围内选择修复粒度，并根据恢复验证决定本轮处理是否结束。图 76 将这一控制关系概括为故障自愈闭环。



图 76: 故障自愈把检测、分级、约束检查、修复与恢复验证连成闭环；修复未能恢复服务目标时，系统重新进入检测与决策过程。

检测信号的准确性决定后续修复是否会被正确触发，因此首先需要明确平台如何判断实例或节点已经失效。

检测：平台如何发现故障 自愈的第一步是发现故障。平台无法直接“知道”一个实例是否还能正常工作，只能通过外部信号推断，这些信号大致分为三类。第一类是主动探测：平台按固定周期向实例发送探测请求（如 K8s 的 Liveness/Readiness Probe、负载均衡的健康检查），根据是否在超时内正确响应判断实例状态。前面场景三中介绍的就绪探针就是这类信号的一种。第二类是被动上报：实例主动向注册中心发送心跳，或在出错时上报异常；一旦心跳超时，注册中心将其从可用列表中剔除。这类信号依赖实例自己“报平安”，适合服务注册发现体系。第三类是指标异常：监控系统持续采集错误率、尾部延迟等指标，当指标越过阈值（如错误率超过 5%、P99 延迟超过 2 秒）时间接推断出下游可能已经故障。这类信号发现的往往不是单个实例的死

活，而是整体服务质量的劣化。三类信号各有盲区：检查内容过浅的主动探测可能只确认进程存在，却没有覆盖关键业务路径；被动上报依赖实例自身仍能通信，指标异常则可能滞后于故障发生。生产系统通常组合使用，而不是只依赖其中一种。

修复：改变执行面上的哪一层对象 检测到故障后，平台的修复动作按影响范围分为由轻到重的三级，每一级改变的是执行面上不同粒度的对象。第一级是进程重启：在原节点上重启容器，适合内存泄漏、死锁这类进程内故障。中断粒度最小（秒级），Pod 的 IP 和调度位置都不变，对应 Liveness Probe 失败后 kubelet 的动作。第二级是实例重建：删除故障 Pod、由控制器在合适的 Node 上重新创建一个，适合节点磁盘写满、局部网络不通这类靠重启进程无法解决的故障。这正是前面调谐循环中“副本数低于期望值就补位”的机制，代价是新 Pod 的 IP 会变化，因此调用方必须通过 Service 而非固定 IP 访问。第三级是节点替换：下线故障节点、申请新机器加入集群，适合物理机宕机、磁盘损坏这类整机故障。这个动作通常由云平台的节点组、实例组自动修复或专门的节点修复机制完成；Cluster Autoscaler 主要根据不可调度 Pod 和节点利用情况调整节点数量，并不等同于故障节点修复器。节点替换耗时最长，且要求节点上的状态已外置，否则数据可能随节点一起丢失。这三级动作并不是孤立的工具，而是平台在不同故障粒度上采用的修复方式：进程级故障就地重启，Pod 失效时由工作负载控制器创建替代实例，节点级故障则由集群与云平台基础设施协同替换机器。它们能够自动进行的共同前提是实例可以被安全替换，也就是前面反复强调的无状态或状态外置。一个把状态留在本地内存或本地磁盘的实例，删掉重建就等于丢数据，平台的自动修复反而会造成损失。

自愈策略的边界与约束 上面三级动作容易给人一种印象：出故障就重启或重建即可。但生产级自愈不是“进程死了就立刻拉起”这么简单，否则可能引发新的问题。最典型的是重启风暴：如果某个依赖（如数据库）临时不可用，导致一批实例同时探测失败，平台若不加约束地同时重启全部实例，重启后它们又会同时涌向尚未恢复的依赖，再次失败、再次重启，系统在“崩溃—重启—再崩溃”之间振荡，始终无法恢复。因此，真实的自愈是一个带约束的决策，而不是无条件的重启。这些约束包括：其一，可用性约束。计划驱逐一个 Pod 前要确认它不是该服务最后的可用副本，否则维护操作会直接导致服务完全中断。K8s 用 PodDisruptionBudget (PDB) 表达“自愿中断期间至少保留多少个可用副本”，主要约束节点排空、集群升级等可计划的驱逐操作。PDB 不能阻止节点突然宕机、进程崩溃等非自愿故障，因此不能替代副本冗余和故障恢复机制。其二，依赖与预热约束。有些实例重启后需要重新加载大体积缓存或建立连接池才能提供服务，这段预热时间内不应被计入可用容量，否则流量会打到尚未就绪的实例上，这正是 Readiness Probe 与 Service 后端列表要守住的边界。其三，下线时序约束。对长连接和有状态服务，修复动作在终止旧实例前需要先排水：停止接收新流量、等待存量请求或会话结束，必要时迁移状态。这与前面安全缩容所采用的“停止新流量、处理存量、再终止实例”是同一顺序。把这些约束合起来看，自愈的本质是在“尽快恢复”和“不因恢复动作制造新故障”之间取平衡。

回到游戏后端：不同服务的自愈代价 同一套检测与修复机制，落到游戏后端的不同服务上，代价差别很大，判断依据仍然是状态特征。对 lobby-api 这类短请求、无状态服务，自愈最

简单：探测失败后删除重建，新 Pod 就绪即自动加入 Service 后端，个别失败请求由客户端重试吸收，玩家几乎无感。对 gateway 和 battle 这类长连接、有状态服务，自愈不能简单地删了重建：网关上承载着大量 WebSocket 长连接，战斗服上运行着正在进行的对局，直接重启会让相关玩家同时断线或掉出对局。这类服务的修复必须配合排水、连接重连或对局状态恢复，平台提供下线时序保证，具体何时可以安全中断由业务逻辑定义。对 redis、postgres 这类有状态存储，自愈的边界最严：它们的数据不能随实例消失，因此不能靠“删除重建”修复，而要依赖数据外置、副本复制和主从切换等机制，平台的自动修复至多是重新调度任务，不能替代数据层面的高可用设计。前面章节介绍的 Deployment（维持副本数）、Service（稳定入口）、Readiness/Liveness Probe（就绪与存活判定）、PDB（约束自愿中断）、preStop 与优雅终止（排水时序）、反亲和（分散故障影响）、Cluster Autoscaler（容量不足时扩展节点），分别在容量调整、计划维护和故障恢复中承担不同职责。明确这些职责后，可以把它们统一到本章的总原则：把失败作为常态纳入系统设计。更高阶的故障场景——有状态服务的一致性恢复、依赖链上的级联故障，以及用熔断（circuit breaker）和自适应限流阻断故障扩散——已经超出本章“如何部署与运维”的范围。

5.4 小结：从标准化交付到弹性运行

本章围绕在线系统进入云环境后面临的环境差异、服务拓扑、负载变化和实例故障，按照“容器—编排—弹性运行”三块建立部署与运维链路，并以第四章的分布式游戏后端验证这些机制。三块内容分别处理运行环境、服务拓扑和持续运行状态，最终目标都是让系统在实例会重启、被替换、重新放置和伸缩的条件下仍能维持服务目标。

第一块是容器。Docker 通过分层构建与摘要锁定，把一次偶然成功的部署变成可复制的制品；镜像、容器、数据卷、隔离机制和容器网络共同保证同一服务可以在不同机器上按相同方式启动。这个阶段解决的是“交付什么，以及换机器后为什么还能一致运行”。

第二块是编排。Compose 把单机上的多容器拓扑写成声明，集群级平台进一步处理跨节点网络、服务发现、放置和副本维护。Kubernetes 用 Pod、Deployment、Service 等对象描述目标状态，再由控制面和调谐循环持续把现实状态拉回目标。这个阶段解决的是“多个服务如何统一部署、互联和长期维护”。

第三块是弹性运行。资源调度决定副本放在哪里，HPA、VPA 与 Cluster Autoscaler 调整应用和基础设施容量，Serverless 为低频短任务提供按需执行方式，故障自愈则在实例或节点损坏后完成检测、修复和恢复验证。这些机制都受 SLO、启动时延、状态归属、下游容量和安全下线条件约束，不能被简化为“自动加机器”或“出错就重启”。

三块内容最终汇聚到云原生系统中的一个基本原则：**为失败而设计**（Design for Failure）。系统设计阶段就要承认机器会宕机、实例会重启、网络会抖动、流量会突增、请求会重复到达。副本冗余、Readiness 闸门、滚动更新、反亲和分散、缩容排水、幂等写入和可观测性，都是把失败纳入平台流程之后形成的工程机制。

读完这一章，至少应该留下几条能带走的判断：镜像要锁内容摘要，入口要和实例解耦，副本数要由平台长期维持，缩容要先排水，核心链路不能轻易归零。第 6 章会把这些判断放到底层

机制里解释：容器隔离、网络虚拟化、伸缩闭环与 Serverless 引擎，都会回到“变化中如何维持稳定”这条主线。

5.5 思考题

1. 团队用 `latest` 标签部署了三台大厅服。某天有人推送了新的同名镜像，导致一台服务器重启后运行新版本，另外两台仍然运行旧版本。请解释这种“版本漂移”是怎样发生的，并说明用镜像摘要（digest）锁定版本为什么能避免这类问题。
2. 周五晚高峰，运维人员用脚本逐台重启进程做热修。脚本执行到一半时网络抖动，导致部分实例已更新、部分实例仍是旧版本。请解释这种中途失败会怎样表现为“有些玩家能登录、有些玩家不能登录”，并说明声明式管理如何把“脚本执行到一半”转化为“现实状态偏离目标状态”来处理。
3. 新部署的 `lobby-api` Pod 进程已经启动，但配置尚未加载完，Redis 连接也没有建立。如果此时没有 Readiness Probe，玩家侧可能看到什么现象？如果 Liveness Probe 设置得过于激进，例如 1 秒内没有响应就重启，又会带来什么问题？请分别说明 Readiness 和 Liveness 守住的是哪两类边界。
4. `lobby-api` 的 CPU 使用率只有 40%，但 P99 延迟已经升到 800ms。如果 HPA 只看 CPU，它会做出什么判断？请列出至少两类“CPU 不高但延迟很高”的原因，例如下游服务变慢、连接池耗尽、锁竞争或队列积压，并提出一个更贴近玩家体验的自定义指标作为扩缩容输入。
5. 战斗服正在缩容，某个 Pod 上还有 5 场正在进行的对局。如果平台直接杀掉进程，玩家侧会看到什么现象？请设计一个“先关门、再清场、最后拉闸”的排水流程，并说明 Readiness Probe、Service 后端摘除和 `preStop` hook 分别在哪一步发挥作用。
6. 给出三种游戏功能：(a) 每日签到奖励发放；(b) 实时对战的帧同步；(c) 赛季结束时的排行榜结算。请分别判断它们是否适合 Serverless，并从连接驻留、状态粘性、冷启动延迟和可重试性四个维度说明理由。
7. 某活动预计在前 10 分钟内把登录 QPS 从 2,000 提升到 8,000。已知一个 `lobby-api` 副本在 P99 延迟可接受的前提下大约能稳定处理 600 QPS，新 Pod 从创建到 Ready 约需 45 秒。请设计一个活动前后的容量策略：基线副本数应如何设置，高峰前是否需要预热，HPA 的扩容速度应如何限制，活动结束后缩容应如何排水。进一步思考：如果只按平均 QPS 配置容量，玩家会在哪些时刻最先感受到排队？
8. 数据库 `postgres` 临时不可用，导致一批 `lobby-api` 实例的探针在几秒内同时失败。如果平台不加约束地立即重启全部失败实例，会出现什么现象（提示：重启后它们仍会同时连向尚未恢复的数据库）？请说明这种“重启风暴”为什么无法靠单纯重启收

敛，为什么 PodDisruptionBudget 不能阻止 Liveness Probe 触发的容器重启，以及区分 Liveness 与 Readiness、加入退避和依赖检查分别能在哪一环降低振荡风险。再进一步：为什么说“能否自愈由架构决定，而不是由平台决定”？

9. 某在线协作系统包含三类组件：处理文档列表查询的无状态 API、维护 WebSocket 会话的实时协作网关，以及每天凌晨执行一次的文档归档任务。请分别选择常驻容器或 Serverless 作为部署方式，并说明选择依据。若实时协作网关需要缩容，还应使用什么指标触发伸缩，怎样完成连接排水？
10. 某模型推理服务的每个 Pod 启动后需要加载模型并预热 90 秒。高峰到来时，CPU 利用率仍然不高，但 GPU 请求队列和 P99 延迟持续上升。请为该服务确定一个可量化的 SLO，选择比 CPU 更合适的伸缩信号，并说明 Readiness Probe、预热容量、HPA 与 Cluster Autoscaler 应如何配合。若只在队列已经积压后才申请新节点，系统会面临什么时间差？